Neural Networks 201 (2026) 108977

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Neural network pruning and simultaneous feature and structure selection

Xinyue Zhang *, Hong Gu, Toby Kenney

Department of Mathematics and Statistics, Dalhousie University, Canada

ARTICLE INFO

Keywords:
Neural network
Feature selection
Model selection
LASSO
Neural network architecture

ABSTRACT

Neural network pruning makes large neural networks more portable and smaller neural networks more interpretable with minimal loss of predictive accuracy. We propose a pruning method that reformulates the neural-network LASSO problem as a standard weighted regression or classification problem with a LASSO penalty. We apply this method starting from a dense neural network structure that includes all possible feed-forward networks as subnetworks. This efficiently removes substantial redundancy. To further refine the network, we develop a second step, which cycles over all remaining links, one at a time, removing those that do not sufficiently improve the model fit.

We demonstrate the effectiveness and stability of our method for both regression and classification problems in four simulation studies. Our method improves both prediction and interpretability, compared with the original dense neural network, and with state-of-the-art neural network pruning methods. Our method also outperforms the state-of-the-art methods across ten real data examples (five regression and five classification).

1. Introduction

Artificial neural networks have proven highly effective across a wide range of real-world applications (Haykin, 1999). Numerous architectures, including feed-forward (Bebis & Georgiopoulos, 1994), recurrent (Medsker & Jain, 2001), and convolutional networks (O'Shea & Nash, 2015), have been developed and extended to increasingly complex structures to improve performance on large-scale data. The complexity of a neural network is determined by both its architecture and the number of input features (Giles & Maxwell, 1987). While deeper and wider networks can approximate more complicated functions, they also require greater computational resources, are harder to train, and tend to generalize poorly (Sabo & Yu, 2008), particularly in high-dimensional settings (Gui et al., 2016).

These challenges are amplified in smaller-sample domains such as biomedical studies, where high-dimensional inputs but limited observations are common. In such cases, feature selection and interpretability are important, making large neural networks less suitable. Methods to remove irrelevant input variables (feature selection) and unnecessary links or hidden nodes (structure selection) can improve effectiveness for small or moderate data sets. Moreover, reduced network structures often better capture the dominant patterns in the data and help avoid overfitting.

Pruning is a widely used approach for removing redundant features or connections that do not contribute to model performance, thereby reducing model size and computational cost (Karnin, 1990). With the rise of increasingly large neural architectures, pruning has become an active research area. However, many pruning techniques rely on heuristic criteria or require predefined sparsity levels, and they often do not explicitly account for how pruning decisions affect model performance during training. In addition, pruning procedures are frequently sensitive to the initial model.

In this paper, we propose a pruning approach that performs simultaneous feature and structure selection. The sparsity level is data-driven, rather than predefined. The resulting reduced architectures provide insight into the structure of the fitted function, while retaining or improving the original predictive accuracy. Our framework follows a backward selection strategy beginning from a dense network that contains all feed-forward networks with H or fewer hidden nodes. This flexible starting point avoids the constraints of predefined layer widths and allows the final architecture to emerge directly from the data.

The pruning procedure consists of two steps. First, pruning at each node is reformulated as a generalized linear model (GLM) in the weights of its incoming connections, enabling the use of LASSO (Tibshirani, 1996) for link selection among these connections. Cycling this LASSO-based step across all internal and output nodes removes many weak connections efficiently and provides a strong initialization for further pruning. Although LASSO-based pruning has appeared previously, our GLM reformulation is, to our knowledge, new and offers substantial computational benefits. Second, we refine the resulting sparse network by evaluating individual remaining links and removing each one if its deletion does not increase the loss beyond a specified tolerance. This

* Corresponding author.
E-mail addresses: xn770547@dal.ca (X. Zhang), hgu@dal.ca (H. Gu), tb432381@dal.ca (T. Kenney).

https://doi.org/10.1016/j.neunet.2026.108977
Received 9 July 2025; Received in revised form 10 February 2026; Accepted 8 April 2026
Available online 13 April 2026

hard-threshold pruning step is time-consuming, but is more informative than previously used heuristic criteria. There is substantial scope to speed up the hard-thresholding step by reusing calculated node values. This would require rewriting the neural network code from scratch to correctly store calculated values, and we have therefore not undertaken this task for this proof-of-concept paper. For a dense network with many hidden nodes and few inputs, this would speed up computation by a factor of 2. For sparser networks, or networks with more input features, the speed-up can be much more. Our two-stage process yields a compact, interpretable, and accurate network with improved generalizability.

Because the method is not tied to a rigid layered structure, it can uncover architectures beyond traditional feed-forward designs, often revealing meaningful feature groupings and hierarchical relationships. In simulations where the true function contains interacting subcomponents, the learned networks visually recover these structures. For real datasets, analyses of cross-layer connections show that the resulting models frequently include substantial cross-layer and direct input-to-output links.

We evaluate the proposed method through extensive simulations and real-data analyses, compare it with existing pruning approaches, and conduct ablation studies to confirm the importance of the second step. Code for all experiments is available at https://github.com/XinyueZ0406/Simultaneous-Feature-and-Structure-Selection-of-Dense-Neural-Network.

The remainder of this paper is organized as follows. Section 1.1 reviews related work on feature and structure selection for neural networks and highlights how our method differs from prior approaches. Section 2 presents the formulation of nodewise backward pruning with a LASSO penalty as a standard weighted regression or classification problem, allowing the use of existing LASSO-penalized GLM procedures. This reformulation applies to other penalties such as SCAD (Fan & Li, 2001), MCP (Zhang, 2010), and Laplace (Wen et al., 2015). We provide analytical solutions for regression and for binary and multi-label classification losses. Section 3 details our two-step pruning algorithm. Section 4 compares the method to state-of-the-art pruning methods for four simulation studies — two regression and one each for binary and multi-label classification. We also examine the interpretability of our learned structure in that section. Section 5 compares our method with competing methods on five regression and five classification datasets. Section 6 concludes the paper.

1.1. Related work on neural network pruning

Neural network pruning removes links, nodes, channels, etc., from an initially over-parameterized network to produce a more compact architecture (Augasta & Kathirvalavakumar, 2013). By selecting relevant features and simplifying the network structure, pruning can improve performance, reduce computational cost, and enhance interpretability.

A wide variety of pruning methods have been developed, differing in timing of pruning; criteria for pruning; and whether they prune individual weights, or groups of weights.

Methods that prune groups of weights are referred to as “structured pruning methods”. The groups may correspond to input features, neurons, or channels (Molchanov et al., 2019). Many methods are based on Group LASSO (e.g. Feng & Simon, 2017; Lemhadri et al., 2021; Wang et al., 2017; Zhang et al., 2019), because of its natural suitability for this problem, but some methods reparametrize the data to allow the use of LASSO, e.g. (Sun et al., 2016). These methods focus on pruning input features, but the group LASSO penalty can apply equally well to select hidden nodes as well Wang et al. (2019, 2018), or to select filters in a convolutional neural network (Oyedotun et al., 2020). For simultaneously selecting input features and hidden nodes, some authors have preferred multi-step LASSO (Fan et al., 2019) and garrote (Wang et al., 2021b) procedures. The few non-LASSO-based approaches to structured pruning include L_0 sparse group penalties (Shao et al., 2022), greedy group-pruning with theoretical guarantees (Rajaraman et al., 2023), and class-wise structured penalties (Liu et al., 2025). The major advantage of structured pruning is that the pruned network is in a form which requires less computation. However, this comes at the cost of less flexibility to adapt to the underlying structure of the data, impairing both performance and interpretability.

The more flexible “unstructured” pruning methods tend to be based on more heuristic measures of link importance. Link weight has been widely used to prune links, e.g. (Frankle & Carbin, 2018; Liu et al., 2024). Extensions, such as movement pruning and sparse-optimization-guided pruning (Sanh et al., 2020; Shi et al., 2024) have shown advantages in performance and robustness. These methods are typically performed iteratively, with a small proportion of links pruned at regular intervals during training. This is referred to as Iterative Magnitude Pruning (IMP) (Frankle & Carbin, 2018), and is still a widely-used method because of its strong empirical performance and simplicity. Gradient-based criteria have the advantage that they can be applied to untrained networks, a class of methods referred to as “Pruning-at-Initialization (PaI)” that sacrifice some predictive performance for computational benefits. PaI is outside the scope of the current paper, but some of the approaches are surveyed in Wang et al. (2021a), and more recent advances include (Pham et al., 2023; Zhao et al., 2025).

A recent development in the field is Dynamic sparse training (DST), which performs both pruning and regrowth of links, to maintain a constant sparsity level. Early work such as SET (Mocanu et al., 2018) regrows a random set of links, while RigL (Evci et al., 2020) improves accuracy and stability by using gradient-based regrowth and has become a competitive state-of-the-art DST method, achieving high sparsity with minimal accuracy loss.

There have been some methods that compromise between structured and unstructured pruning. Approaches include sparse group LASSO (Scardapane et al., 2016), and other penalties (Alemu et al., 2019; Bao et al., 2022) that induce sparsity on the groups (nodes in these cases), providing structured pruning, and also induce within-group sparsity on the weights, giving unstructured pruning.

To our knowledge, our proposed method is the first application of LASSO to unstructured pruning of neural networks. The key innovation here is the iterative application of LASSO to groups of input connections to a fixed node. This greatly improves stability over attempts to apply LASSO to all weights simultaneously, while retaining the advantages of LASSO over the more heuristic methods widely used for unstructured pruning. Our method follows an iterative pruning-after-training paradigm but also shares some procedural similarity with DST in that pruned links are not permanently removed until the end of the pruning process. This flexibility reduces sensitivity to initialization and training dynamics. The hard-thresholding step is another important innovation of our method, that better aligns the pruning procedure with the final objective, and allows our method to achieve higher sparsity levels and better interpretability.

2. Penalized GLM fitting for neural network backwards node-wise pruning

2.1. Dense neural network and notation

As a starting point to building our network selection method, we build an initial neural network with a maximal set of connections subject to no feedback loops. We refer to this initial network as the Dense Neural Network (DNN). The structure is similar to the DenseNet (Huang et al., 2017) structure in CNN with one node in each hidden layer. However, there is no convolution structure in our network and all weights are freely estimated.

Fig. 1 shows the dense neural network architecture for a regression problem when there is only one neuron in the output layer. Let P be the number of features, let H be the number of hidden layers or equivalently the number of hidden nodes, and let Q be the number of output nodes

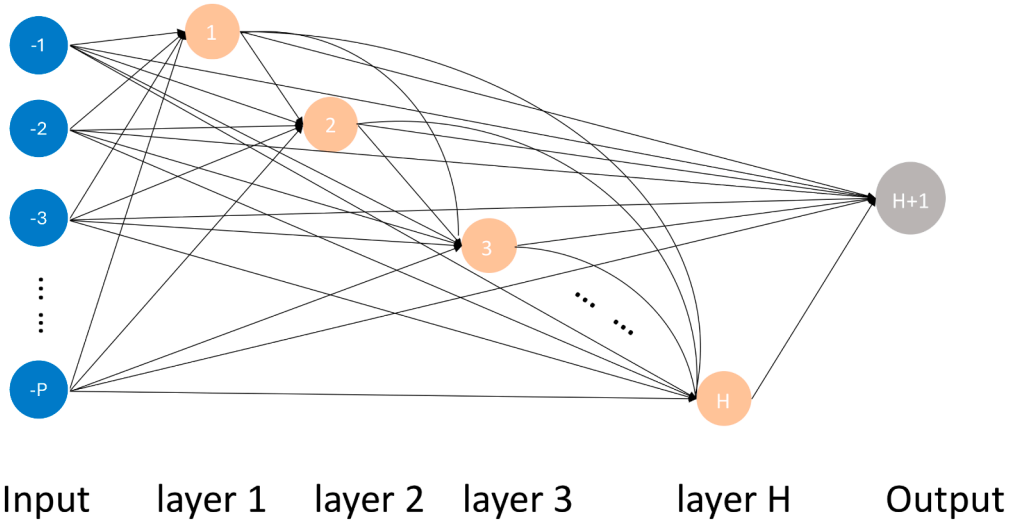


Fig. 1. Dense neural network (DNN) architecture.

(usually $Q = 1$, but for some problems such as multi-label classification or multivariate regression, there may be multiple output nodes). Suppose the training sample is defined by $\{x_i, y_i\}_{i=1}^n$, where x_i is the ith input sample and a vector of dimension P, and y_i is the corresponding target output. To simplify the notation, we sometimes drop the subscripts and use x and y to refer to any observation's input and output. The weights are arranged in a weight matrix of dimension $(P + H) \times (H + Q)$:

$$W = \begin{pmatrix} W_{-P,1} & W_{-P,2} & \cdots & W_{-P,H} & W_{-P,H+1} & \cdots & W_{-P,H+Q} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ W_{-1,1} & W_{-1,2} & \cdots & W_{-1,H} & W_{-1,H+1} & \cdots & W_{-1,H+Q} \\ 0 & W_{1,2} & \cdots & W_{1,H} & W_{1,H+1} & \cdots & W_{1,H+Q} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & W_{H-1,H} & W_{H-1,H+1} & \cdots & W_{H-1,H+Q} \\ 0 & 0 & \cdots & 0 & W_{H,H+1} & \cdots & W_{H,H+Q} \end{pmatrix}$$

For convenience, we use negative indices for the weights corresponding to the input layer, and the associated input, so that the indices for hidden nodes run from 1 to H. In a slight abuse of terminology, we will refer to this matrix as upper triangular, meaning that when extended to a $(P + H + Q) \times (P + H + Q)$ matrix by appending zero columns on the left and zero rows below, the resulting matrix is upper triangular. We also express this matrix according to its column vectors as $W = (W_1, W_2, \ldots, W_H, W_{H+1}, \ldots, W_{H+Q})$, where the last columns W_{H+1} to W_{H+Q} are the weight vectors connecting all input nodes and all hidden layers to the output layers. For other columns of W, for example, the hth column W_h is the weight vector connecting all input nodes and all previous $h - 1$ hidden layers with the hth hidden layer. Let the bias vector be $b = (b_1, b_2, \ldots, b_h, \ldots, b_H, b_{H+1}, \ldots, b_{H+Q})^T$, where b_h is the bias of the hth layer.

Let a_h be the activation functions of the node in the hth layer. (Usually all hidden nodes have the same activation function, but there is no cost to this more general definition.) We let N_{ih} denote the pre-activation value of node h for the ith observation, where $h \in \{1, \ldots, H + Q\}$. Let the overall activation function be given by

$$a(x, N)_{ij} = \begin{cases} x_{ij} & \text{if } j < 0 \\ a_j(N_{ij}) & \text{if } 0 < j \leqslant H \end{cases}$$

The pre- and post-activation values of the nodes are then given by the solution to $N = a(x, N)W + 1b^T$. Because W is upper triangular, this can be calculated recursively, with N_{ij} calculated from x_i and $(N_{il})_{l<j}$.

We let g be the output activation function, that takes the vector of output node values $(N_{i(H+1)}, \ldots, N_{i(H+Q)})$ as input and produces the corresponding vector of predicted values $(\hat{y}_{i1}, \ldots, \hat{y}_{iQ})$ as output. We therefore have $\hat{y} = g(N_{>H})$, where $N_{>H}$ is the matrix of the last Q columns of N, and we apply g row-wise on the matrix $N_{>H}$ of all output node values.

As for a standard feed-forward neural network, we train the weights W and the bias b by back-propagation to optimize the chosen loss function. The advantage of using the DNN as a starting point is that every neural network structure with a total of H or fewer hidden nodes without feedback can be embedded in the DNN structure, so can be achieved after the pruning.

2.2. Backwards nodewise LASSO selection

Regularization is widely used to avoid over-fitting of the model to the data, and LASSO is one of the most popularly used regularization methods. LASSO minimizes a penalized loss function with the L^1 penalty for the model parameters, which achieves the purpose of shrinkage and selection simultaneously. It is well known that LASSO estimates are biased, thus we are not using LASSO penalized estimates as the final estimates for the neural network weights. The LASSO penalty used here is for selection of input variables and neural network weights and nodes only. Other penalties such as SCAD (Fan & Li, 2001), MCP (Zhang, 2010) or Laplace (Wen et al., 2015) penalties could also be used in our methods. We choose LASSO due to its computational speed.

We first develop a method using LASSO to perform shrinkage on the weights of all links with endpoint h. We then apply this method by cycling through all nodes h, starting with the output layer and working backwards, selecting the links with endpoint h. It is possible to penalize all weights at the same time using a penalized loss function, however the results are less stable than the backwards nodewise selection procedure proposed here, because of the strong interactions between connecting weights. By restricting attention to blocks of weights with a common endpoint, we avoid these interactions, and obtain more stable results. We refit DNN for 10 epochs starting from the sparse weight matrix after LASSO selection. This DNN refit procedure is to find a local minimum for the loss function around the sparse weight matrix. We repeat the LASSO selection cycle and DNN refit until convergence. The detailed procedure is given in Section 3.

In more detail, suppose we have performed LASSO on all links with endpoint after h, so that the weight matrix is sparse in the columns after h. As in the previous section, we let N denote the pre-activation values of each node for each observation, using the current weights W and bias values b. We now want to shrink the weights W_{jh} for $j < h$. We will use an asterisk $(^*)$ to denote values based on the shrunken weights, so W^* denotes the weight matrix with these new values and b^* denotes the updated bias values. N_{ik}^* denotes the pre-activation node values for node

k and data point i using the shrunken values W^*, and $\hat{y}_i^* = g(N^*_{i,>H})$ is the final estimate using these shrunken values. We have the constraint $W^*_{jk} = W_{jk}$ whenever $k \neq h$, and W^*_{jh} are to be estimated for $j < h$ (and by the upper triangular constraint on W, $W^*_{jh} = 0$ for all $j \geqslant h$). For $k < h$, we have $N^*_{ik} = N_{ik}$ and for $k \geqslant h$, N^*_{ik} is given by solving $N^* = a(N^*)W^* + 1(b^*)^T$.

When $h = H + q$ is an output node, we have $N^*_k = N_k$ for all $k \neq h$, and N^*_h is a linear function of W^*_h. Therefore, the optimization is a standard penalized Generalized Linear Model (GLM), and there are a number of existing packages to implement LASSO.

For a hidden node h, the output nodes are a nonlinear function of W^*_h. In order to convert the problem to a standard GLM, we will take a linear approximation for the values of the output nodes as a function of W^*_h, using the first order Taylor series. As a function of W^*_h, $N^*_{i(H+q)}$ depends only on the value of N^*_{ih}, so we let ${}_hf_{ij}(N^*_{ih})$ be the value of N^*_{ij} for any W^*_h with the corresponding value of N^*_{ih}. The Taylor series then gives

$$N^*_{iq} \approx N_{iq} + ({}_hf_{iq})'(N_{ih})(N^*_{ih} - N_{ih}) \tag{1}$$

The function ${}_hf_{ij}$ is very complicated and depends on all the nodes $a(N)_{il}$ for $l < h$ and all the weights W_{lm} for $m > h$. Since (1) is a linear function of W^*_h, standard software can usually be used to implement LASSO for this approximation. The derivative $({}_hf_{iq})'$ is widely used in estimating the weights of a neural network, and is typically calculated using a recursive procedure, based on the chain rule.

More specifically, we let ${}_hC_{ij} = \left.\frac{dN^*_{ij}}{dN^*_{ih}}\right|_{N^*_{ih}=N_{ih}}$, for any $j \geqslant h$. We are interested in the particular case where $j = H + q$ is an output node; in this case, we have $({}_hf_{i(H+q)})'(N_{ih}) = {}_hC_{i(H+q)}$. Now by the chain rule, ${}_hC_{ij}$ can be calculated recursively by

$${}_hC_{ih} = 1$$
$${}_hC_{ik} = \sum_{j=h}^{k-1} {}_hC_{ij} W_{jk} a'(N_{ij}) \qquad \text{for } k > h$$

Thus, we have $\hat{y}_i^* \approx g(N_{i,>H} + {}_hC_{i,>H}(N^*_{ih} - N_{ih})))$. We therefore optimize the penalized loss function $\sum_{i=1}^n L(y_i, g(N_{i,>H} + {}_hC_{i,>H}(N^*_{ih} - N_{ih}))) + \lambda \sum_{j<h} |W^*_{jh}|$ where $L(y, \hat{y})$ is the loss for a particular estimator $\hat{y}$, when the observed value of the response variable is y.

2.2.1. Regression problem

For regression problems, we consider squared error loss. The identity activation function is traditionally used — i.e. $g(x) = x$ for all x. The loss function without penalty is given by

$$\begin{aligned}
\text{Loss} &= \sum_i (\hat{y}_i^* - y_i)^2 \\
&\approx \sum_i \{N_{i(H+1)} - y_i + {}_hC_{i(H+1)}[N^*_{ih} - N_{ih}]\}^2 \\
&= \sum_i ({}_hC_{i(H+1)})^2 \left\{ \left[\frac{y_i - N_{i(H+1)}}{{}_hC_{i(H+1)}} + N_{ih} \right] - N^*_{ih} \right\}^2 \\
&= \sum_i ({}_hC_{i(H+1)})^2 \left[R_{ih} - \left(\sum_{\substack{j=-P,\\ j\neq 0}}^{h-1} a(N)_{ij} W^*_{jh} + b^*_h \right) \right]^2
\end{aligned}$$

where $R_{ih} = \frac{y_i - N_{i(H+1)}}{{}_hC_{i(H+1)}} + N_{ih}$ is the value of the N^*_{ih} such that the linear approximation is equal to the observed value, i.e. ${}_hf_{i(H+1)}(N_{ih}) + ({}_hf_{i(H+1)})'(N_{ih})(R_{ih} - N_{ih}) = y_i$. When ${}_hC_{i(H+1)} = 0$, we define $R_{ih} = N_{ih}$.

This is exactly the loss function for a weighted linear regression with weights $({}_hC_{i(H+1)})^2$ and observed values R_{ih} for the response variable. Thus, the penalized likelihood can be optimized using standard software for fitting LASSO. The penalized loss function is given by:

$$\text{Loss} = \sum_i \left({}_hC_{i(H+1)}\right)^2 \left[R_{ih} - \left(\sum_{\substack{j=-P,\\ j\neq 0}}^{h-1} a(N)_{ij} W^*_{jh} + b^*_h \right) \right]^2 + \lambda \sum_{\substack{j=-P,\\ j\neq 0}}^{h-1} |W^*_{jh}|$$

For multivariate regression problems, there will be multiple output nodes. It is straightforward to modify the above loss function to a sum over all output nodes for the weighted regression loss term.

2.2.2. Binary classification problem

For binary classification problems, there is a single output node, usually with a logistic activation function $g(x) = \frac{1}{1+e^{-x}}$. The cross-entropy or log-likelihood loss

$$\text{Loss} = -\sum_i (y_i \log(\hat{y}_i^*) + (1 - y_i)\log(1 - \hat{y}_i^*))$$

is generally used for such problems.

We have $\hat{y}_i^* = \frac{1}{1+e^{-N^*_{i(H+1)}}}$. For the hth hidden node, we have that

$$\begin{aligned}
N^*_{i(H+1)} &= {}_hf_{i(H+1)}(N^*_{ih}) \\
&\approx (N_{i(H+1)}) + {}_hC_{i(H+1)}(N^*_{ih} - N_{ih})
\end{aligned}$$

This means that

$$\begin{aligned}
\hat{y}_i^* &\approx \frac{1}{1 + e^{-N_{i(H+1)} - {}_hC_{i(H+1)}(N^*_{ih} - N_{ih})}} \\
&= \frac{1}{1 + e^{({}_hC_{i(H+1)})N_{ih} - N_{i(H+1)} - {}_hC_{i(H+1)}N^*_{ih}}} \\
&= \frac{1}{1 + e^{-(\alpha_i + b^*_h({}_hC_{i(H+1)}) + (W^*_h)^T({}_hC_{i(H+1)}a(N)_i))}}
\end{aligned}$$

where

$$\alpha_i = N_{i(H+1)} - {}_hC_{i(H+1)}N_{ih}.$$

This is a logistic regression with offset α_i, with no intercept, and with predictor matrix $(\text{diag}({}_hC_{(H+1)}))\begin{pmatrix}1 & a(N)_{<h}\end{pmatrix}$. Thus, we can find W^*_h and b^*_h using L^1-penalized logistic regression. The first coefficient b^*_h is not shrunk.

2.2.3. Multi-label classification problem

For a Q-class classification problem, we have a network with Q output nodes, with pre-activation values $N_{i(H+1)}, \ldots, N_{i(H+Q)}$. We let $\hat{y}_i^* = [\hat{y}^*_{i1}, \hat{y}^*_{i2}, \ldots, \hat{y}^*_{iQ}]$ be the vector of predicted probabilities of each class for the ith observation. These predicted probabilities are given by

$$\begin{aligned}
\hat{y}^*_{ik} &= g(N^*_{i(H+1)}, \ldots, N^*_{i(H+Q)})_k \\
&= \frac{e^{N^*_{i(H+k)}}}{\sum_{l=1}^Q e^{N^*_{i(H+l)}}}
\end{aligned}$$

The loss function is

$$\text{Loss} = -\sum_i \sum_{k=1}^Q y_{ik} \log(\hat{y}^*_{ik})$$

When $h = H + q$ is one of the output nodes, only $N^*_{i(H+q)}$ depends on W^*_{H+q}, with $N^*_{i(H+k)} = N_{i(H+k)}$ for all $k \neq q$. Thus,

$$\begin{aligned}
\hat{y}^*_{iq} &= \frac{e^{N^*_{i(H+q)}}}{e^{N^*_{i(H+q)}} + \sum_{k\neq q} e^{N_{i(H+k)}}} \\
&= \frac{1}{1 + e^{-(O_{iq} + b^*_{H+q} + (W^*_{(H+q)})^T a(N)_i)}}
\end{aligned}$$

where $O_{iq} = -\log\left(\sum_{k\neq q} e^{N_{i(H+k)}}\right)$ is a constant for the current LASSO step. We will show that the shrunken estimated probabilities $\{\hat{y}^*_{ik} | k \neq q\}$ for the other classes are proportional to the original estimated probabilities $\{\hat{y}_{ik} | k \neq q\}$, and therefore, for this particular LASSO step, the overall loss function is equivalent to a binary classification loss.

For $k \neq q$, we have

$$\hat{y}^*_{ik} = \frac{e^{N_{i(H+k)}}}{e^{N^*_{i(H+q)}} + \sum_{l\neq q} e^{N_{i(H+l)}}}$$

The estimated conditional probability $P(Y = k|Y \neq q)$, given by

$$\frac{\hat{y}^*_{ik}}{\sum_{l\neq q}\hat{y}^*_{il}} = \frac{e^{N_{i(H+k)}}}{\sum_{l\neq q} e^{N_{i(H+l)}}} = \frac{\hat{y}_{ik}}{\sum_{l\neq q}\hat{y}_{il}}$$

does not depend on $W^*_{(H+q)}$. Furthermore, since $\sum_{k=1}^{Q}\hat{y}^*_{ik} = 1$, we have

$$\begin{aligned}\hat{y}^*_{ik} &= \frac{\hat{y}^*_{ik}}{\sum_{l\neq q}\hat{y}^*_{il}}\sum_{l\neq q}\hat{y}^*_{il}\\ &= \frac{\hat{y}_{ik}}{\sum_{l\neq q}\hat{y}_{il}}(1-\hat{y}^*_{iq})\end{aligned}$$

If we let ${}_qP_{ik} = \frac{\hat{y}_{ik}}{\sum_{l\neq q}\hat{y}_{il}}$ and $L_{iq} = \sum_{k\neq q}^{Q} y_{ik}\log\left({}_qP_{ik}\right)$, then the loss function is

$$\begin{aligned}\text{Loss} &= -\sum_i\left(y_{iq}\log\left(\hat{y}^*_{iq}\right) + \sum_{k\neq q} y_{ik}\log(\hat{y}^*_{ik})\right)\\ &= -\sum_i\left(y_{iq}\log\left(\hat{y}^*_{iq}\right) + \sum_{k\neq q} y_{ik}\log\left({}_qP_{ik}(1-\hat{y}^*_{iq})\right)\right)\\ &= -\sum_i\left(y_{iq}\log\left(\hat{y}^*_{iq}\right) + (1-y_{iq})\log\left(1-\hat{y}^*_{iq}\right) + L_{iq}\right)\end{aligned}$$

Thus, since L_{iq} does not depend on W^*_{H+q}, the LASSO fitting for this multi-class classification reduces to a logistic regression GLM, where the response is an indicator for Class q.

When h is an internal node, all output nodes depend on W^*_h, but we can still approximate the loss as a series of binary classification problems. The key is to rewrite the loss function

$$\begin{aligned}\text{Loss} &= -\sum_i\sum_k y_{ik}\log(\hat{y}^*_{ik})\\ &= -\sum_i\left(y_{i1}\log(\hat{y}^*_{i1}) + (1-y_{i1})\sum_{q=2}^{Q} y_{iq}\log(\hat{y}^*_{iq})\right)\\ &= -\sum_i\left(y_{i1}\log(\hat{y}^*_{i1}) + (1-y_{i1})\sum_{q=2}^{Q} y_{iq}\log\left((1-\hat{y}^*_{i1})\frac{\hat{y}^*_{iq}}{1-\hat{y}^*_{i1}}\right)\right)\\ &= -\sum_i\left(y_{i1}\log(\hat{y}^*_{i1}) + (1-y_{i1})\log(1-\hat{y}^*_{i1}) + \sum_{q=2}^{Q} y_{iq}\log\left(\frac{\hat{y}^*_{iq}}{1-\hat{y}^*_{i1}}\right)\right)\end{aligned}$$

Then repeatedly applying the same rewriting to the inner sum, we get

$$\begin{aligned}\text{Loss} = -\sum_i\Bigg(&y_{i1}\log(\hat{y}^*_{i1}) + (1-y_{i1})\log(1-\hat{y}^*_{i1})\\ &+ y_{i2}\log\left(\frac{\hat{y}^*_{i2}}{1-\hat{y}^*_{i1}}\right) + (1-y_{i1}-y_{i2})\log\left(1-\frac{\hat{y}^*_{i2}}{1-\hat{y}^*_{i1}}\right)\\ &+ y_{i3}\log\left(\frac{\hat{y}^*_{i3}}{1-\hat{y}^*_{i1}-\hat{y}^*_{i2}}\right)\\ &+ (1-y_{i1}-y_{i2}-y_{i3})\log\left(1-\frac{\hat{y}^*_{i3}}{1-\hat{y}^*_{i1}-\hat{y}^*_{i2}}\right)\\ &+ \dots\\ &+ y_{i(Q-1)}\log\left(\frac{\hat{y}^*_{i(Q-1)}}{\hat{y}^*_{i(Q-1)}+\hat{y}^*_{iQ}}\right) + (1-\dots-y_{i(Q-1)})\log\left(1-\frac{\hat{y}^*_{i(Q-1)}}{\hat{y}^*_{i(Q-1)}+\hat{y}^*_{iQ}}\right)\Bigg)\end{aligned}$$

That is, our loss can be viewed as the total loss from a series of binary classification problems. We can therefore rewrite the loss as a binary classification loss by stacking the data appropriately. We first stack the original data, with the response variable an indicator for the first class; then we stack all observations not in Class 1 with the response an indicator for Class 2; then all observations not in Classes 1 or 2, with responsean indicator for Class 3, and so on until we append the observations from Classes $Q-1$ and Q, with response an indicator for Class $Q-1$.

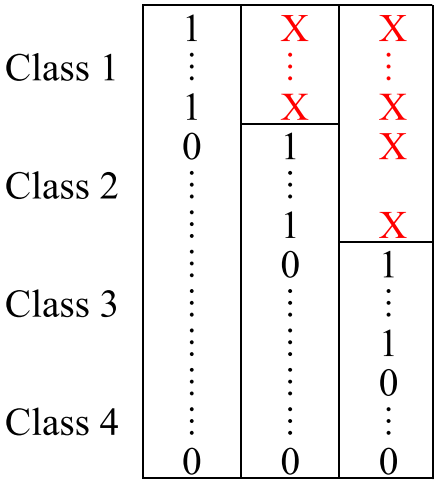


Fig. 2. The stacking procedure for multi-class classification. The original one-hot Y matrix consists of one indicator vector for each class. The entries marked X are deleted, and the remaining entries in each column are stacked into a single response vector.

More formally, let the classes be numbered 1 to Q; let observation i belong to Class number c_i; and let $C = \{(i,j)|j \leqslant c_i, 1 \leq i \leq n\}$ be the set of all remaining entries in Fig. 2. We will stack the elements of C with increasing order for the index j. For every $(i,j) \in C$, we will let y_{ij} be the response variable. For each class j, we will approximate $\text{logit}\left(\frac{\hat{y}^*_{ij}}{\sum_{k=j}^{Q}\hat{y}^*_{ik}}\right)$ by a linear function $(W^*_h)^T\beta_{(i,j)} + b^*_h\gamma_{(i,j)} + \beta_{0(i,j)}$ for some vector $\beta_{(i,j)}$ and some scalars $\gamma_{(i,j)}$ and $\beta_{0(i,j)}$. Then we can estimate W^*_h and b^*_h by a logistic regression with response $(y_{ij})_{(i,j)\in C}$ and predictor vectors $(\beta_{(i,j)}, \gamma_{(i,j)})_{(i,j)\in C}$ and offset $\beta_{0(i,j)}$.

We rewrite the logistic probability:

$$\begin{aligned}&\text{logit}\left(\frac{\hat{y}^*_{iq}}{\hat{y}^*_{iq}+\hat{y}^*_{i(q+1)}+\dots+\hat{y}^*_{iQ}}\right)\\ &= \text{logit}\left(\frac{e^{N^*_{i(H+q)}}}{e^{N^*_{i(H+q)}} + e^{N^*_{i(N+q+1)}} + \dots + e^{N^*_{i(H+Q)}}}\right)\\ &= \log\left(\frac{e^{N^*_{i(H+q)}}}{e^{N^*_{i(H+q+1)}} + \dots + e^{N^*_{i(H+Q)}}}\right)\end{aligned}$$

A first-order Taylor approximation about N_{ih} gives

$$\begin{aligned}&\log\left(\frac{e^{N^*_{i(H+q)}}}{e^{N^*_{i(H+q+1)}} + \dots + e^{N^*_{i(H+Q)}}}\right) - \log\left(\frac{e^{N_{i(H+q)}}}{e^{N_{i(H+q+1)}} + \dots + e^{N_{i(H+Q)}}}\right)\\ &= N^*_{i(H+q)} - N_{i(H+q)} - \log\left(e^{N^*_{i(H+q+1)}} + \dots + e^{N^*_{i(H+Q)}}\right)\\ &\quad + \log\left(e^{N_{i(H+q+)}} + \dots + e^{N_{i(H+Q)}}\right)\\ &\approx ({}_hC_{i(H+q)})(N^*_{ih} - N_{ih})\\ &\quad - \left(({}_hC_{i(H+q+1)})({}_{>q}P_{i(q+1)}) + \dots + ({}_hC_{i(H+Q)})({}_{>q}P_{iQ})\right)(N^*_{ih} - N_{ih})\\ &= \left\{{}_hC_{i(H+q)} - ({}_hC_{i(H+q+1)})({}_{>q}P_{i(q+1)}) - \dots - ({}_hC_{i(H+Q)})({}_{>q}P_{iQ})\right\}\\ &\quad (N^*_{ih} - N_{ih})\end{aligned}$$

where ${}_{>k}P_{ij} = \frac{\hat{y}_{ij}}{\sum_{l>k}\hat{y}_{il}}$ is the conditional probability $P(Y = j|Y > k)$. Thus we have

$$\log\left(\frac{e^{N^*_{i(H+q)}}}{e^{N^*_{i(H+q+1)}} + \dots + e^{N^*_{i(H+Q)}}}\right) \approx (W^*_h)^T\beta_{iq} + b^*_h\gamma_{iq} + \beta_{0(iq)}$$

where

$$\gamma_{iq} = {}_hC_{i(H+q)} - ({}_hC_{i(H+q+1)})({}_{>q}P_{i(q+1)}) - \cdots - ({}_hC_{i(H+Q)})({}_{>q}P_{iQ})$$
$$\beta_{iq} = \gamma_{iq} a(N)_i$$
$$\beta_{0(iq)} = \log\left(\frac{e^{N_{i(H+q)}}}{e^{N_{i(H+q+1)}} + \cdots + e^{N_{i(H+Q)}}}\right) - \gamma_{iq} N_{ih}$$

Thus, we can estimate W_h^* and b_h^* using a binary classification LASSO on this stacked data. That is, rows of the predictor matrix correspond to pairs $(i,q) \in C$, and the entries in the (i,q)th row consist of the entries in the vector β_{iq}, and the value γ_{iq}. The response variable for the (i,q)th entry is y_{iq}, and there is an offset term $\beta_{0(iq)}$. The fitted coefficients in this logistic regression are W_h^* and b_h^*.

2.2.4. Tuning parameter

For LASSO selection, λ is selected automatically based on minimizing cross-validated generalization error. Instead of choosing the value $\lambda_{\min}$ that minimizes cross-validated error, it is a common practice to choose the largest λ value for which the cross-validated error is within one standard deviation of the minimum. This value, denoted λ_{1se} results in a sparser model with similar performance on cross-validated data. Since our application of LASSO is only a first step to be followed by deleting additional links, it is important not to make the selected network too sparse at this stage. This suggests that $\lambda_{\min}$ may be preferable to λ_{1se}. We compare the performance of both methods in the simulation studies and real data analyses.

3. Simultaneous feature and NN structure selection based on backwards LASSO pruning

The backwards nodewise LASSO pruning procedure described in the last Section can be used for pruning any neural network without feedback. However we hope to find an optimum network structure so that the graph of the resulting neural network structure can illustrate the shape of the function it approximates. Thus our feature and structure selection algorithm starts from the fitted dense neural network (DNN), and includes a two-step selection procedure.

The first step is the backwards LASSO pruning (soft threshold) which cycles between LASSO selections and DNN fittings to drop redundant features and links on a large scale to obtain a sparser model. In the LASSO selections, starting from the output node(s) backwards towards hidden nodes, one LASSO selection acts on one column of the weight matrix. If the estimated coefficients are shrunken to zero, the corresponding weights are set to zero. Otherwise, the corresponding weight retains the original value from the basic model. After each LASSO selection for one node, we recalculate the output matrix for all nodes that need to be updated based on the new weight matrix. When one sweep of LASSO selection is finished, all columns of the weight matrix have been updated once. We then perform ten epochs of back-propagation of DNN to readjust all weights and update the basic model. This step uses the zero weights fitted by LASSO as starting values, but results in a less sparse weight matrix. The purpose of this step is to safeguard against an edge being wrongly deleted by a single LASSO optimization, and to readjust the weights to better reflect the selected network structure. We repeat this cycle of LASSO selection and DNN fitting several times until model convergence. The result is a weight matrix with many zero values. Finally the connections with fitted zero weights in the final LASSO steps are dropped. We drop any node (input or hidden) which has either no path from any input nodes, or no path to any output node. We call the resulting model the LASSO shrunken model.

The second step is to further remove links one by one from the LASSO shrunken model by a hard threshold method. This step is necessary because although LASSO selection can drop a large number of redundant connections, it is well known that LASSO variable selection has high false positive rates. We start with the smallest weight from the LASSO shrunken model, set the weight to zero and compute the deterioration in objective loss function by setting the single weight to zero with all other weights retaining their current values. If the increase in loss function is less than a chosen cut-off θ, then the weight is set to zero, otherwise, the weight is reset to the original fitted value. After this, the next weight is considered. After cycling through all the weights, perform an epoch of back-propagation to readjust all weights of the LASSO shrunken model, then repeat the above procedure until model convergence. In consequence, a final sparse model architecture with reduced features and connections can be produced. In order to improve performance, we refit this final sparse model after dropping all connections with zero weights. We use stabilization of the validation loss within an error range of 0.00001 as the criterion for convergence of the model. This complete algorithm is presented in Algorithm 1 and illustrated in Fig. 3.

Algorithm 1: HT-LASSO pruning (LASSO-only pruning — Steps 1–10)

Data: $X_{\text{tr}} = \{x_i, y_i\}_{i=1}^n$, $x_i \in \mathbb{R}^p$, no. of hidden layers H, cut-off θ.

1 Train a DNN with H hidden layers and record it as DNN_{base}.
2 Compute W and N matrix from DNN_{base}.
3 **for** h *in* $[H+Q, H+Q-1, \ldots, 1]$ **do**
4 Perform LASSO selection on the W_h, denote it as β
  if $\beta_j = 0$. **then**
5 $W_{j,h} = 0$
6 **else**
7 $W_{j,h} = W_{j,h}$
8 Update DNN_{base} with the zero weights
9 Perform ten extra epochs for DNN_{base} to readjust all weights in W again.
10 Repeat 3 to 9 until convergence, finishing with 8 to get the network NN_{red}.
11 obj_{base} = loss of NN_{red}; sort weights $W = \{W_{(1)} < W_{(2)} < W_{(3)} < \cdots\}$.
12 **for** $W_{(i)}$ *in* W **do**
13 set $W_{(i)} = 0$ and obtain new reduced model NN'_{red}
14 obj_{new} = objective loss function of NN'_{red}
15 diff = obj_{new} - obj_{base}
16 **if** *diff* $< \theta$ **then**
17 $w_{(i)} = 0$
18 $\text{obj}_{base} = \text{obj}_{new}$
19 $\text{NN}_{red} = \text{NN}'_{red}$
20 Perform an extra epoch for NN_{red} to readjust all weights again.
21 Repeat 12 to 20 until model convergence.
22 Refit the final reduced neural network.

It is convenient to describe the resulting network structure in terms of layers. These are not layers in the traditional feed-forward neural network layer sense where all connections are between adjacent layers. In our reduced network, it is possible for connections to remain between arbitrary layers. We can define the layers inductively. The first layer consists of all nodes whose only inputs are from input nodes; the second layer consists of all nodes not in the first layer whose only connections are from input nodes and nodes in the first layer, and so on.

The cut-off value θ needs to be tuned in our proposed approach. The larger θ, the sparser the model. To ensure the sparsity of model and high accuracy simultaneously, we use a validation set to tune the value θ. In this method, we divide the training samples into two parts: training set and validation set. We choose the value of θ which minimizes the objective loss function on the validation set. We could use cross validation to improve the selection of θ, but because of the computational cost of our method, we prefer to use a single validation set.

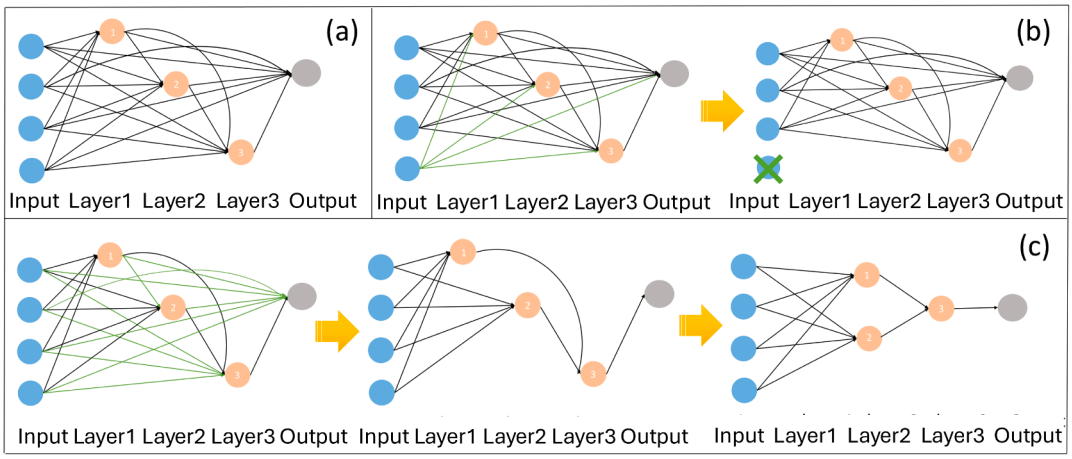


Fig. 3. Structure selection algorithm. (a) The initial DNN structure with 4 inputs and 3 hidden nodes. (b) All connections involving the fourth input feature are removed, so the feature is also removed. (c) When the connection between hidden nodes 1 and 2 is removed, the nodes become part of a single layer in the selected structure. In this example, the final structure is a feed-forward network with two hidden layers.

4. Simulations

4.1. Simulation design

We evaluate our method across four simulation settings: two regression scenarios, one binary classification task, and one three-class classification task. Each dataset contains eight predictors: $X_1, \ldots, X_5 \sim N(0,1)$ independently, and $(X_6, X_7, X_8) \sim N(0,1)$ but correlated with X_1, X_2, and X_3 with coefficients $(0.8, 0.7, 0.2)$, respectively. The response depends only on X_1–X_5, with (X_2, X_3) and (X_4, X_5) forming two functional groups.

Regression simulation 1.

$$Y = 0.2(X_1 - 2)^2 - 0.4(X_2 - X_3)^2 + 0.2e^{X_4+X_5} + \epsilon, \quad \epsilon \sim N(0,1).$$

Regression simulation 2.

$$Y = \frac{1}{1 + \left(1.6\left(\frac{X_1^2}{3} - 1\right)\sin(X_2X_3) + 0.16X_4X_5\right)^3} + \epsilon, \quad \epsilon \sim N(0,2).$$

Binary classification. The response $Y \sim \text{Bernoulli}(p(X))$ with

$$p(X) = \frac{1}{1 + e^{-(0.2(X_1-2)^2 - 0.4(X_2-X_3)^2 + 0.2e^{X_4+X_5})}}.$$

Multiclass classification. We generate three latent scores

$$N_1 = 0.4(X_1 - 2)^2 - 0.8(X_2 - X_3)^2 + 0.4e^{X_4+X_5},$$
$$N_2 = -0.3(X_1 - 2)^2 - 0.4(X_2 - X_3)^2 + 0.8e^{X_4+X_5},$$
$$N_3 = 0.5(X_1 - 2)^2 - 0.5(X_2 - X_3)^2 - 0.4e^{X_4+X_5},$$

with class probabilities $p_i = e^{N_i}/(e^{N_1} + e^{N_2} + e^{N_3})$.

Apart from Regression Simulation 2, each replicate consists of 2,000 training samples, 1,000 validation samples, and 100,000 test samples, and we generate 100 replicates. We analyze each replicate starting from a DNN with $H = 10$ and $\theta \in \{0.1, 0.05, 0.01, 0.005, 0.001, 0.0005, 0.0001, 0\}$. For Regression Simulation 2, due to the complexity of the model, each replicate contains 8,000 training samples, 4,000 validation samples, and 100,000 test samples, with a total of 15 replicates. We examine models with $H \in \{10, 20, 30\}$ and $\theta \in \{1, 0.5, 0.1, 0.05, 0.01, 0.005, 0.001, 0\}$.

In all studies, including the real data analysis in Section 5, the data are normalized before training. The neural network uses ReLU activations in the hidden layers and is optimized with stochastic gradient descent (SGD) using a step-based learning rate schedule that halves the learning rate every 20 epochs, starting from 0.1.

We compare our approach with two widely used unstructured pruning methods: Iterative Magnitude Pruning (IMP) (Frankle & Carbin, 2018) and RigL (Evci et al., 2020). All methods operate on the same initial dense neural network and use identical data splits to ensure a fair comparison. IMP performs post-training pruning through iterative magnitude-based weight removal with weight rewinding to the initial dense parameters, while RigL implements dynamic sparse training by periodically updating sparse connectivity via drop-and-grow operations. Following standard practice, we run IMP with a fixed number of pruning rounds and vary the target sparsity across a broad grid (5%, 10%, 20%, …, 90%, 95%, 99%). For RigL, we adopt the implementation in Evci et al. (2020), training sparse networks from scratch at each specified sparsity level using the prescribed update schedule.

4.2. Simulation results

Fig. 4 presents the mean squared errors (MSE) with their standard errors for the two regression simulations, and Fig. 5 displays the cross-entropy loss and classification accuracy for the two classification simulations. Our method involves two hyperparameters, λ and θ, which are selected using a validation set in each experiment. To evaluate the contribution of each component of the method, we perform an ablation study comparing the full pruning procedure—LASSO followed by hard-thresholding (HT-LASSO)—with a reduced version that applies only the LASSO step (i.e., the output after Step 10 of Algorithm 1). This allows us to isolate the effect of the hard-thresholding step on sparsity, predictive performance, and computational efficiency.

For IMP and RigL, sparsity is treated as a fixed control variable rather than a tunable hyperparameter. We therefore evaluate these methods across a range of sparsity levels and highlight the sparsity level with lowest validation loss. Detailed numerical results for Figs. 4 and 5, together with comparisons using λ_{1se} and λ_{min} for both the LASSO-only and HT-LASSO variants under fixed θ, are provided in Tables A.11, A.13, A.14, and A.16 in the supplementary appendices.

From Figs. 4 and 5, our method consistently improves upon the original DNN. During the LASSO stage, a substantial number of links, noise features, and internal nodes are removed with minimal loss in predictive accuracy. The full HT-LASSO procedure further enhances performance: the hard-thresholding step reliably outperforms one-step LASSO pruning, indicating that LASSO alone does not fully eliminate redundant connections or features. However, the LASSO step alone achieves good MSE and sparsity in a much shorter time than the full HT-LASSO method, so may be used in cases where training time is of critical importance. Compared with IMP and RigL, our approach achieves lower test losses and

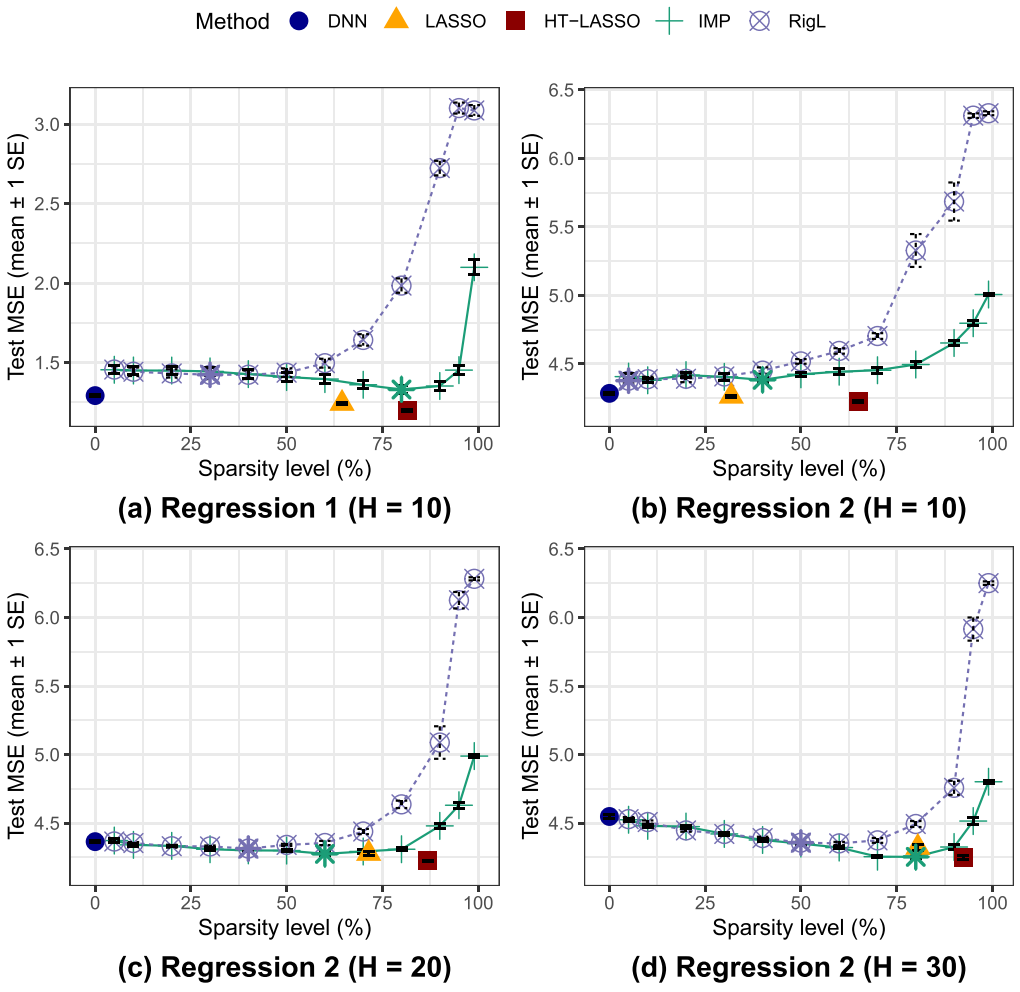


Fig. 4. Test data MSE versus sparsity level for two regression simulations. The larger symbols for IMP and RigL show the sparsity level that achieves lowest MSE on validation data. Error bars indicate ±1 standard error.

smaller standard errors with sparser models, demonstrating improved generalization and greater stability across replicates. Notably, even the LASSO-only variant surpasses both baselines.

Comparing different numbers, H, of hidden nodes in Fig. 4, the baseline DNN increasingly overfits, especially at $H = 30$. In contrast, our pruned models effectively control overfitting and improve prediction accuracy, consistently outperforming IMP and RigL while producing even sparser networks. They also exhibit stronger stability and generalization, with the hard-thresholding step providing refinement beyond LASSO alone.

Supplementary Tables A.11, A.13, A.14, and A.16 show a clear train–test gap for the original DNN, which is substantially reduced after applying our pruning method. Pruning improves generalization by mitigating overfitting, yielding a much smaller discrepancy between training and test performance. Although λ_{1se} produces a sparser model and performs well immediately after the LASSO step, it provides less benefit during hard-thresholding because under-pruning can be corrected in the second stage, whereas overly aggressive pruning cannot. Predictive accuracy is stable across a wide range of θ values, indicating low sensitivity to this threshold. In terms of computation, models based on $\lambda_{\min}$ generally require more time, and smaller θ values likewise increase cost.

Table 1 shows the frequency with which each combination $(\lambda_{1se}, \theta,)$ or $(\lambda_{\min}, \theta)$ is selected by HT-LASSO. We see that the selected values for θ are fairly tightly grouped, with most values close to the values of θ that produce optimal results with fixed θ (see Supplementary Tables A.11, A.13, A.14, and A.16). λ_{1se} is chosen more often for the simpler simulation scenarios, while $\lambda_{\min}$ is chosen more frequently for the more complicated scenarios.

Table 2 reports the average running time of all methods across the simulation settings. The LASSO-only procedure is considerably faster than IMP, while RigL and HT-LASSO are much slower. Within HT-LASSO, the hard-thresholding stage is more time-consuming than the initial LASSO step, partly because our current implementation has not been optimized. There are possible approaches to substantially reduce the running time. In cases where time is a limiting factor, the LASSO-only procedure could be used to gain most of the pruning benefits of our method with a competitive running time. The framework is general and applicable to pruning in a wide variety of neural network architectures.

Table 3 summarizes the feature and structure selection results for both HT-LASSO and the LASSO-only variant. HT-LASSO consistently identifies all five true variables while rarely selecting noise variables, whereas the LASSO-only approach retains several irrelevant features. This explains its weaker predictive performance and highlights the importance of the second hard-thresholding stage for accurate feature and structure recovery. HT-LASSO procedure also substantially reduces model complexity, eliminating approximately 80–95% of the links in the original DNN, with small standard errors across replicates indicating stable performance.

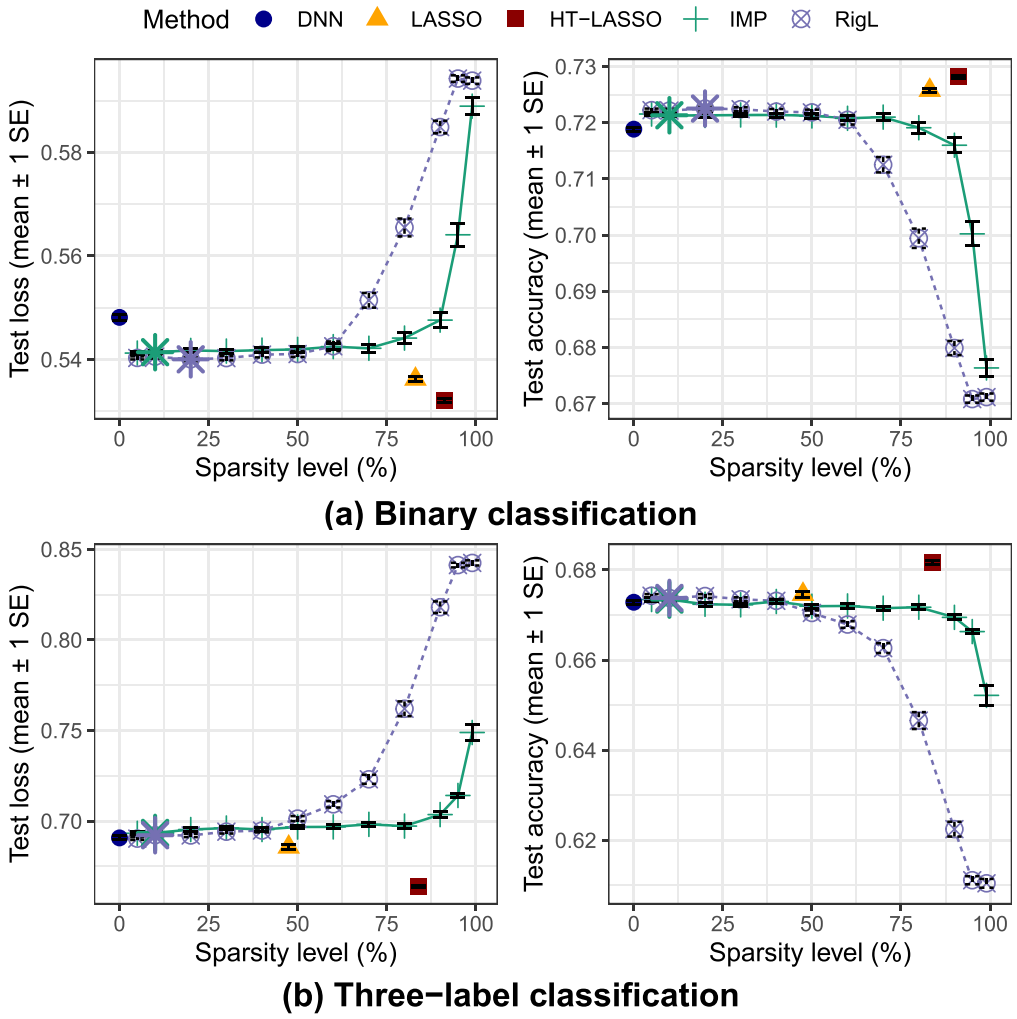


Fig. 5. Test data cross-entropy loss and accuracy versus sparsity level for two classification simulations. The larger symbols for IMP and RigL show the sparsity level that achieves lowest loss on validation data. Error bars indicate ±1 standard error.

Table 1
Frequency with which each combination $(\lambda_{\min}, \theta)$ or (λ_{1se}, θ) is selected by HT-LASSO among 100 replicates for Regression simulation 1 and for the two classification settings, and among 15 replicates for Regression simulation 2.

Cut-off value	Regression 1		Binary		Three-label		Reg. 2 (H = 10)		Reg. 2 (H = 20)		Reg. 2 (H = 30)	
	$\lambda_{\min}$	λ_{1se}	$\lambda_{\min}$	λ_{1se}	$\lambda_{\min}$	λ_{1se}	$\lambda_{\min}$	λ_{1se}	$\lambda_{\min}$	λ_{1se}	$\lambda_{\min}$	λ_{1se}
$\theta = 0$	1	1	0	8	0	0	0	0	0	0	0	0
$\theta = 0.0001$	2	4	0	2	1	0	–	–	–	–	–	–
$\theta = 0.0005$	1	4	0	4	1	0	–	–	–	–	–	–
$\theta = 0.001$	1	5	4	11	7	0	0	1	0	2	0	2
$\theta = 0.005$	4	10	13	29	70	0	7	2	2	6	2	2
$\theta = 0.01$	11	24	3	26	21	0	3	1	2	2	3	4
$\theta = 0.05$	6	18	0	0	0	0	0	1	0	1	0	2
$\theta = 0.1$	2	6	0	0	0	0	0	0	0	0	0	0
$\theta = 0.5$	–	–	–	–	–	–	0	0	0	0	0	0
$\theta = 1$	–	–	–	–	–	–	0	0	0	0	0	0

Table 2
Average running time (second).

		DNN	LASSO	HT-LASSO	IMP	RigL
Regression 1		44.37	364.04	12103.74	866.21	6090.69
Regression 2	$H = 10$	90.03	1045.42	4758.83	2065.57	21689.42
	$H = 20$	120.69	1480.99	4750.73	3061.69	38832.77
	$H = 30$	154.16	2228.10	5793.66	4264.84	60108.80
Binary classification		29.51	710.34	2223.34	902.84	3022.62
Three-label classification		30.66	923.30	8533.56	918.37	3608.46

In Regression Simulation 2, when $H = 10$, the method retains most of the hidden nodes, as the underlying function requires more than ten nodes. As H increases to 20 or 30, the number of selected nodes stabilizes at roughly 11 — the amount needed to fit the model for the given sample size. The number of layers and links selected by HT-LASSO is similar across values of H, demonstrating that the method can begin with a large architecture and automatically select an appropriate network size. In contrast, the LASSO-only variant fails to remove many noisy variables and retains more connections as H grows. The subse-

Table 3
Structure comparisons across simulation settings. Values are averages over all replicates; standard errors (SE) are shown in parentheses.

Setting		Method	Input	Noise	Node	Layer	Link
Regression 1		LASSO	5.00 (0.00)	2.83 (0.05)	6.86 (0.15)	3.41 (0.19)	54.71 (2.89)
		HT-LASSO	5.00 (0.00)	0.79 (0.11)	6.53 (0.12)	2.16 (0.11)	28.45 (1.30)
	H = 10	LASSO	5.00 (0.00)	3.00 (0.00)	9.40 (0.19)	6.53 (0.52)	105.87 (7.66)
		HT-LASSO	5.00 (0.00)	0.20 (0.11)	9.40 (0.19)	4.00 (0.31)	54.53 (2.78)
Regression 2	H = 20	LASSO	5.00 (0.00)	2.93 (0.07)	10.87 (0.96)	6.73 (1.21)	114.13 (22.72)
		HT-LASSO	5.00 (0.00)	0.47 (0.19)	10.00 (0.55)	3.53 (0.34)	52.47 (4.12)
	H = 30	LASSO	5.00 (0.00)	3.00 (0.00)	12.23 (1.47)	7.92 (1.50)	144.77 (33.70)
		HT-LASSO	5.00 (0.00)	0.54 (0.27)	11.23 (1.08)	4.00 (0.25)	57.00 (5.29)
Binary Classification		LASSO	5.00 (0.00)	1.94 (0.11)	3.57 (0.16)	1.94 (0.13)	26.06 (2.09)
		HT-LASSO	5.00 (0.00)	0.28 (0.07)	3.28 (0.11)	1.41 (0.06)	13.56 (0.61)
Three-label Classification		LASSO	5.00 (0.00)	3.00 (0.00)	7.90 (0.20)	5.54 (0.20)	99.76 (2.73)
		HT-LASSO	5.00 (0.00)	0.34 (0.11)	6.72 (0.19)	2.80 (0.12)	30.70 (1.14)

quent hard-thresholding step corrects this over-selection, yielding final network sizes that remain similar across different H values.

More detailed comparisons of feature and structure selection—covering HT-LASSO, the LASSO-only variant, both $\lambda_{\min}$ and λ_{1se} in the LASSO step, and several fixed values of θ—are provided in Supplementary Tables A.12, A.15, and A.17. These tables show that using $\lambda_{\min}$ yields a less sparse model after the LASSO step, but the subsequent hard-thresholding stage prunes the network to a sparsity level comparable to that obtained with λ_{1se}. The hard-thresholding step also reduces variability in the selected structures, as indicated by smaller standard errors. When λ_{1se} is used, the resulting models are highly sparse and nearly identical across different θ values, which helps explain their weaker predictive performance. For large cutoff values ($\theta > 0.05$), pruning removes nearly all variables and connections; the resulting networks rely almost entirely on bias terms, leading to a substantial drop in predictive accuracy.

Fig. 6 presents examples of the reduced network structures obtained after applying HT-LASSO for one dataset from each simulation setting. All structures exhibit the correct feature-grouping pattern: inputs X_2 and X_3, as well as X_4 and X_5, connect through shared hidden nodes, while X_1 contributes primarily through separate pathways. This aligns with the design of the simulated data-generating mechanism.

For the regression setting, the resulting network displays a clear hierarchical organization. The structure for Regression Simulation 2 is more complicated than the other settings, reflecting the fact that the true function is more complex. The binary classification setting yields the sparsest structure, where each input group connects to a single hidden node, and the hidden nodes then connect to the output node, with additional direct input–output connections. In the multi-class classification case, more cross-layer connections among hidden nodes appear, but the input-grouping patterns and the overall hierarchical structure of the network remain evident.

We also assessed sensitivity to initialization by running one replicate of Regression Simulation 1 with 100 random seeds. Because the seed affects both initial weights and stochastic gradients, this offers a direct test of stability. As shown in Supplementary Appendix Table A.18, the results are highly consistent across seeds.

5. Real data analyses

We evaluate our method on eight data sets from the UCI Machine Learning Repository (Asuncion & Newman, 2007), including four regression, two binary classification, and two multi-label classification tasks. To further examine performance and scalability in more data-rich settings, we also include two moderate-scale UCI data sets: the Credit data set (binary classification) and the Energy data set (regression). Across all real-data experiments, we compare our method with two widely used pruning baselines—IMP and RigL—using a common initial DNN architecture to ensure a fair comparison.

Table 4
Regression data sets summary.

	Data Size	Input Features
Concrete Compressive Strength	1030	8
Housing	506	13
Combined Cycle Power Plant	9568	4
Airfoil Self Noise	1503	5

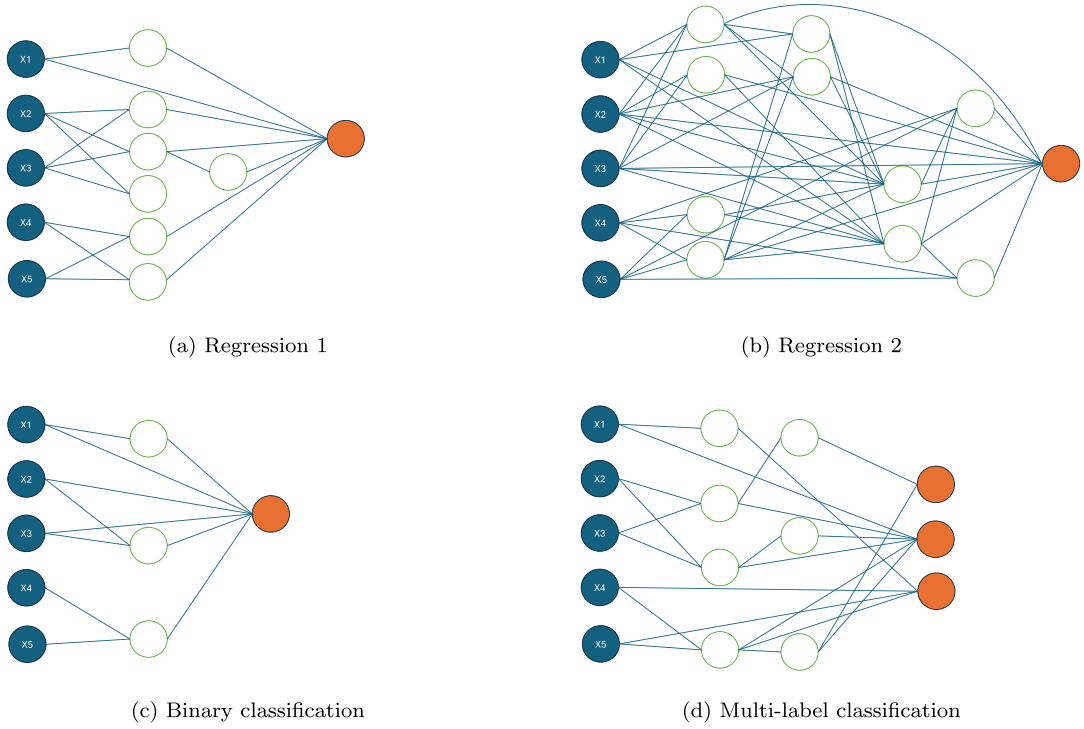
5.1. Regression examples

Table 4 summarizes the four regression data sets. For each data set, we fit DNNs with 20 and 30 hidden layers to compare their performance. Each data set is randomly split into 80% training and 20% testing, and 10-fold cross-validation is conducted on the training portion to determine the optimal cut-off value. This random splitting is repeated seven times, and the average root mean squared error (RMSE) is reported in Fig. 7.

Fig. 7 shows that, compared with IMP and RigL, both HT-LASSO and the LASSO-only variant achieve markedly lower test RMSE across all data sets, demonstrating superior generalization on real regression tasks. Although increasing H offers modest improvements for IMP and RigL, their test errors remain consistently higher than those of the proposed approaches. Moreover, while IMP and RigL suffer substantial performance degradation after pruning, our method maintains strong predictive accuracy while removing a large proportion of connections, pruning more than 40% of links for $H = 20$ and over 55% for $H = 30$.

For the Housing data set, nonzero values of θ are selected more frequently, indicating that a sparser structure is appropriate. For the remaining three data sets, $\theta = 0$ is selected most often, suggesting that a more complex network is suitable for these cases relative to the simulation settings. The LASSO-only variant performs comparably to the HT-LASSO while being much more computationally efficient and significantly faster than both IMP and RigL.

Table 5 summarizes both the selected network structures and the cross-layer connections in our reduced models. These models retain all input features but include far fewer hidden layers than a full DNN. Compared with the simulations, the selected structures are less sparse, suggesting that the underlying regression functions are more complex and involve richer interactions among predictors. This greater complexity may also help explain why the improvements in prediction accuracy were smaller than those observed in the simulations. As H increases, the models include more hidden nodes and deeper connectivity, leading to improved predictive performance. The right portion of the



(a) Regression 1 (b) Regression 2

(c) Binary classification (d) Multi-label classification

Fig. 6. Examples of reduced network structures from applying HT_LASSO for one dataset from each simulation setting.

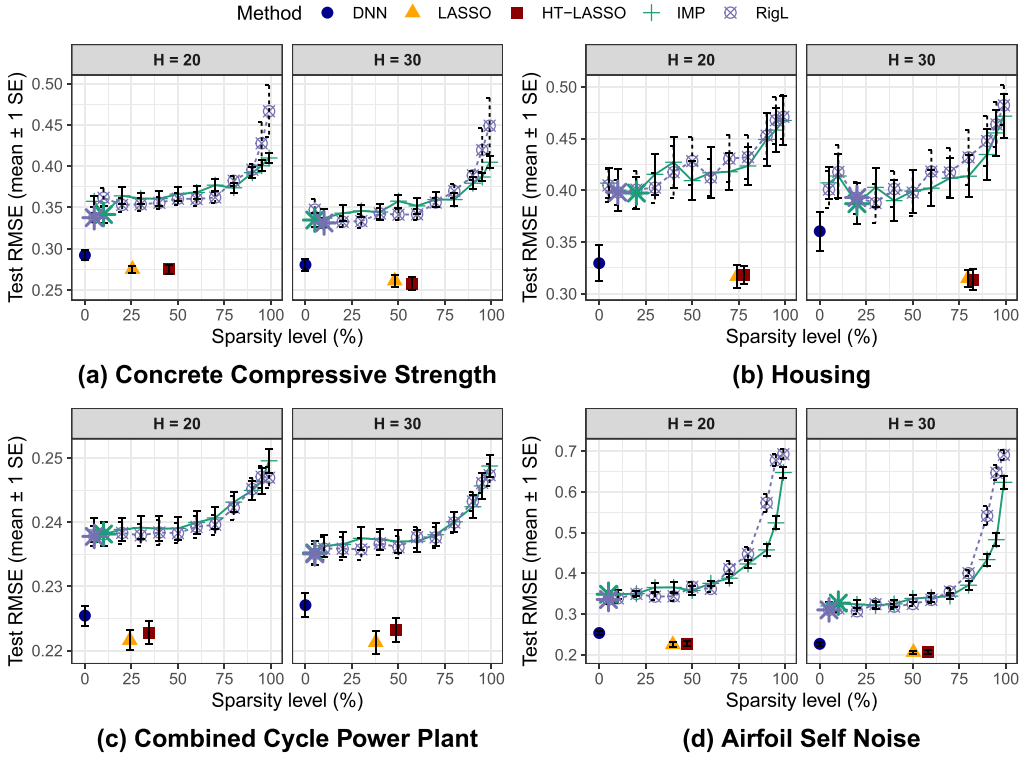


Fig. 7. Test data RMSE versus sparsity level for four real data regression examples. The larger symbols for IMP and RigL show the sparsity level that achieves lowest MSE on cross-validation data. Error bars indicate ±1 standard error.

Table 5
Summary of selected structures and cross layer connections for real data regression analyses.

Data	Model	H	Average selected structures				Cross layer connections			
			input	node	layer	link	node–node	node–output	input–node	input–output
Concrete Compressive Strength	LASSO	$H = 20$	8	19.3	15.3	297	109	17.7	131	8
		$H = 30$	8	25.6	15	387	150	24	156	7.86
	HT-LASSO	$H = 20$	8	19.1	11.7	218	72.6	16.7	103	7.71
		$H = 30$	8	25.4	11.6	316	106	22.6	125	7.57
Housing	LASSO	$H = 20$	13	10.3	4.57	130	7.71	9.14	58.3	13
		$H = 30$	13	14.6	6.71	182	17.3	13.3	95.4	12.9
	HT-LASSO	$H = 20$	13	10.3	4	113	5.57	9.14	46.1	12.3
		$H = 30$	13	14.4	5.14	159	12.1	12.6	75.6	11.9
Combined Cycle Power Plant	LASSO	$H = 20$	4	19.7	15	239	118	18	72.1	4
		$H = 30$	4	28.3	19.7	386	218	26	101	4
	HT-LASSO	$H = 20$	4	19.4	13.4	207	89.9	17.4	67.3	4
		$H = 30$	4	27.7	16.7	318	159	24.3	93.4	4
Airfoil Self Noise	LASSO	$H = 20$	5	17.2	13.6	203	81.9	15.9	75.2	5
		$H = 30$	5	25.2	17.7	323	158	23.5	101	5
	HT-LASSO	$H = 20$	5	17.9	12.4	177	66.4	14.9	65.7	5
		$H = 30$	5	24.7	16	272	121	21	88	5

Table 6
Classification data sets summary.

		Data Size	Input Features
Binary Classification	Liver	345	6
	Sonar	208	60
Multi-label Classification	Iris	150	4
	Vertebral	310	6

table further shows that the reduced models include extensive cross-layer connections—such as hidden-to-hidden, hidden-to-output, input-to-hidden, and input-to-output links — deviating substantially from the architecture of standard feed-forward neural networks. Notably, Nearly all inputs connect directly to the output node across all data sets, indicating that the models capture linear effects explicitly. In contrast, standard feed-forward neural networks typically represent such linear components only implicitly through combinations (or convolutions) of nonlinear activation functions.

5.2. Classification examples

All four classification data sets are summarized in Table 6, including two multi-label data sets with three classes. We consider $H = 10$ and $H = 20$ for comparison. Performance is measured using cross-entropy loss and prediction accuracy. The output layer uses the sigmoid activation for binary classification and softmax for multi-label classification. Each data set is randomly split into 70% training and 30% testing, with 10-fold cross-validation on the training set to select cut-off values. This procedure is repeated ten times, and we report the averages in Fig. 8.

Fig. 8 shows the average test loss and accuracy over ten runs for all classification data sets. Both the LASSO-only and HT-LASSO models generally achieve lower test loss and higher accuracy than the full DNN, except on the Sonar data set. LASSO-only performs similarly to HT-LASSO, with only slightly higher loss. Compared with IMP (Frankle & Carbin, 2018) and RigL (Evci et al., 2020), our reduced models provide better predictive performance across the four data sets and yield substantially sparser networks.

As H increases, the DNN attains much lower training loss but higher test loss, indicating substantial overfitting. In contrast, our reduced models keep training and test losses close and maintain stable loss and accuracy across all H values.

For the Sonar data set, the full DNN achieves both the highest test loss and the highest test accuracy. The large cross-entropy loss typically indicates miscalibration due to overfitting. Although the model often assigns high probabilities to the true class, the probabilities assigned are much higher than would be justified, resulting in high loss function. This is common for flexible high-dimensional models with small sample size. Pruning mitigates this overfitting, but the reduced accuracy indicates that the pruning may also remove informative links.

Table 7 summarizes the selected structures and cross-layer connections for the reduced classification models. Compared with IMP and RigL, our reduced models exhibit substantially higher sparsity, with more than 90% of links removed. As H increases, the numbers of retained inputs, hidden nodes, and hidden layers all decrease, indicating the selection of increasingly sparse networks. These sparser architectures yield slightly lower test loss. The LASSO-only and HT-LASSO models show nearly identical behavior: HT-LASSO removes fewer than five additional links and retains almost the same hidden-layer structure, suggesting that the LASSO step alone is sufficient to identify an effective reduced network for these data sets.

Except for the Sonar data set, nearly all input features are retained. For Sonar, which has many more predictors than the other data sets, the final model keeps only about half of the inputs. This reduced input set helps explain why the full DNN has higher accuracy than the reduced models for this dataset.

Compared with the regression data sets, the classification problems yield fewer hidden nodes and a simpler hidden-layer structure, resulting in fewer cross-layer connections overall. In the Iris and Vertebral data sets, most selected input features connect directly to the output node, indicating a strong linear component and suggesting that standard feed-forward neural networks may not be optimal for these tasks. Furthermore, the very limited number of hidden-to-hidden connections implies a clearer hierarchical structure within the hidden layers.

Table 8 reports the running times for all above real data analyses. Consistent with the simulation studies, the LASSO-only pruning procedure is highly efficient, whereas the hard-thresholding step is considerably more time-consuming. This additional cost is mainly due to our non-optimized implementation for fitting neural networks with a fixed sparse structure and for evaluating the objective function in larger networks. At present, these steps involve substantial redundant computation, which contributes to the longer running time.

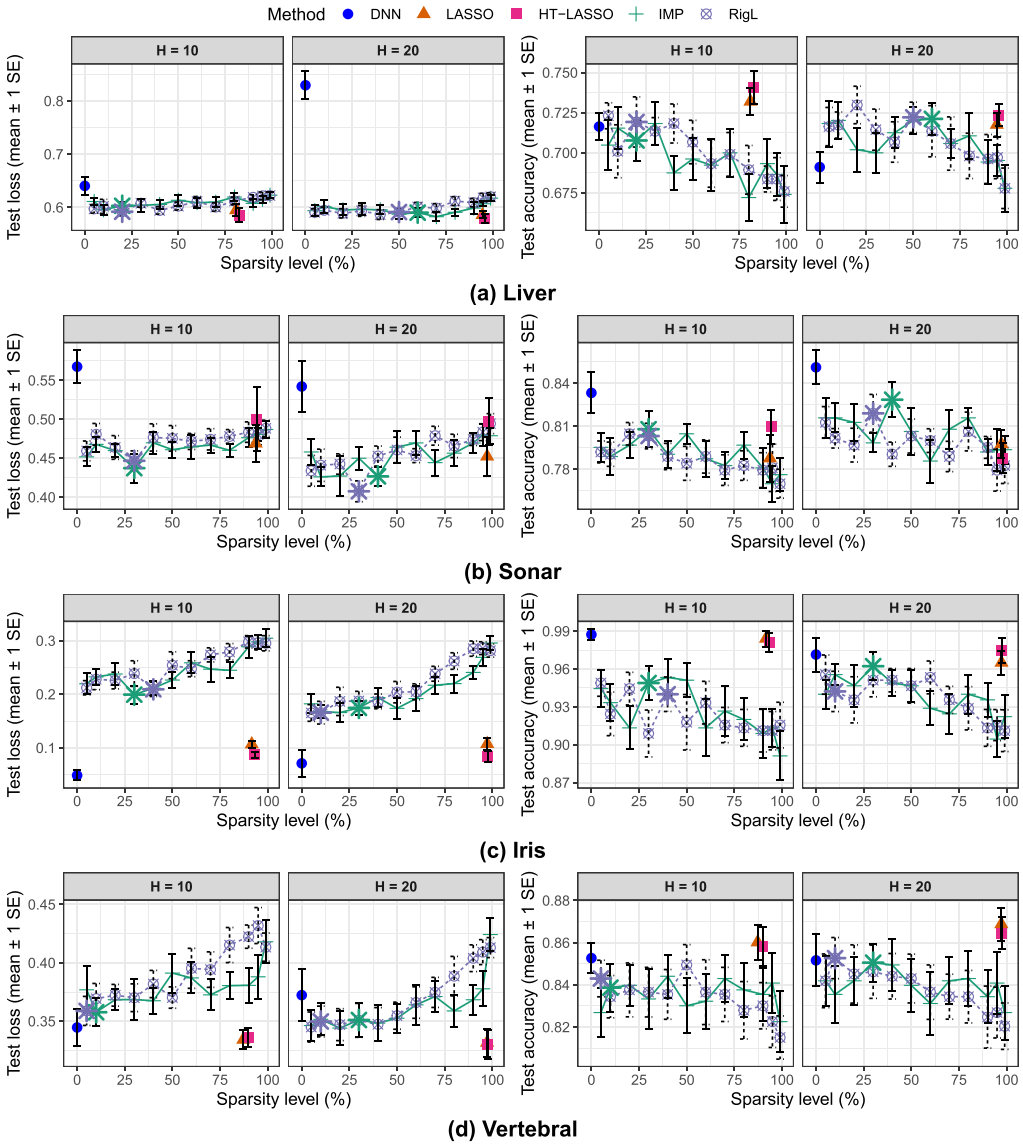


Fig. 8. The average test loss and accuracy versus sparsity level for four real data classification examples. The larger symbols for IMP and RigL show the sparsity level that achieves lowest loss on cross-validation data. Error bars indicate ±1 standard error.

5.3. Moderate-Scale datasets

The Credit data set contains information on customer default payments in Taiwan and is used as a binary classification task to distinguish defaulting from non-defaulting clients. It consists of 30,000 observations and 23 original predictors, which expand to 33 variables after one-hot encoding. The Energy data set comprises 10-minute resolution measurements collected over approximately 4.5 months, including indoor temperature and humidity readings from a wireless sensor network, energy consumption, and external weather variables. We analyze this data set as a regression problem aimed at predicting appliance energy usage in a low-energy building. It contains 19,735 observations and 27 predictors.

For both data sets, the observations are randomly partitioned into training (70%), validation (10%) and testing (20%) sets. All experiments are repeated ten times under different random splits, and results are reported as averages across replications. Given the larger sample sizes of these data sets, we set $H = 50$ in all analyses.

Fig. 9 displays the relationship between sparsity level and test performance for all methods. For the Credit data set, both LASSO-only and HT-LASSO achieve their best performance at very high sparsity levels (approximately 98%), indicating that only a small subset of connections is sufficient for accurate prediction. In contrast, IMP and RigL attain their optimal performance at intermediate sparsity levels (around 40%–60%). The full DNN exhibits the poorest test performance, reflecting substantial overfitting when using an architecture with $H = 50$. All pruning-based methods effectively mitigate this issue.

For the Energy data set, IMP and RigL select very low sparsity levels (around 5%) that slightly outperform our method, whereas our approach delivers competitive performance with sparsity level exceeding 90%. The stronger performance of IMP and RigL in this setting may be due to the increased H. In the simulations and smaller real data sets, these methods benefitted more from an increase in H.

The more detailed numerical results together with the running time for these two data set is given in Table 9.

Table 10 summarizes the average selected structures and cross-layer connections identified using the LASSO-only procedure and HT-LASSO. For both datasets, the LASSO-only step already produces sparse architectures with competitive predictive performance, leaving limited room for further improvement through the hard-thresholding step in terms of

Table 7
Summary of selected structures and cross-layer connections for real data classification problems.

Data	Model	H	Average selected structures				Cross layer connections			
			input	node	layer	link	node–node	node–output	input–node	input–output
Liver	LASSO	H = 10	6	3.5	2.5	25.2	0.5	1.8	7.8	5.8
		H = 20	6	2.6	1.9	19.1	0.3	1.0	5.3	3.5
	HT-LASSO	H = 10	6.0	3.5	2.4	22.9	0.3	1.9	7.0	5.1
		H = 20	5.8	2.6	1.7	14.9	0.2	1.0	3.5	2.8
Sonar	LASSO	H = 10	33.7	1.6	1.3	45.7	0.0	0.3	7.3	17.3
		H = 20	31.0	1.2	0.9	40.2	0.0	0.2	3.4	11.8
	HT-LASSO	H = 10	31.8	1.6	1.3	41.0	0.0	0.3	5.9	15.6
		H = 20	24.8	1.0	0.8	29.4	0.0	0.2	2.7	7.6
Iris	LASSO	H = 10	4.0	0.9	0.9	9.2	0.0	0.2	0.4	2.4
		H = 20	4.0	0.6	0.6	8.4	0.0	0.1	0.9	1.0
	HT-LASSO	H = 10	3.4	0.8	0.8	7.4	0.0	0.2	0.4	1.2
		H = 20	3.7	0.6	0.6	7.6	0.0	0.1	0.9	0.9
Vertebral	LASSO	H = 10	5.6	2.5	2.0	16.9	0.6	1.1	2.2	5.8
		H = 20	4.8	1.8	1.6	9.8	0.0	0.1	0.0	4.2
	HT-LASSO	H = 10	5.2	2.5	1.9	13.4	0.1	1.0	0.9	5.0
		H = 20	4.6	1.8	1.6	8.9	0.0	0.1	0.0	3.9

Table 8
Running time (seconds) for real data sets across different models.

Data set	H	DNN	LASSO	HT-LASSO	IMP	RigL
Concrete Compressive Strength	20	68.77	436.08	30280.14	996.56	7125.07
	30	94.45	921.73	41544.69	1256.96	10421.67
Housing	20	38.60	461.98	17792.75	657.48	4419.11
	30	47.22	920.88	19820.19	1495.80	4214.76
Combined Cycle Power Plant	20	218.70	1031.70	87233.38	5776.33	44101.36
	30	475.32	1711.11	93190.04	8315.29	70736.40
Airfoil Self Noise	20	102.26	475.25	30062.79	1112.86	11279.89
	30	129.00	1018.15	43982.71	1440.20	16188.23
Liver	10	19.53	168.02	1108.08	462.55	1538.58
	20	25.18	270.26	865.16	540.80	2613.40
Sonar	10	8.95	344.96	1784.27	390.04	1147.76
	20	10.35	441.96	1024.74	500.10	1947.32
Iris	10	14.55	174.15	269.40	359.72	677.92
	20	15.56	316.67	244.78	416.13	1169.69
Vertebral	10	15.31	223.98	429.08	435.83	1128.18
	20	20.12	389.72	206.23	503.03	1878.06

Table 9
Model performance for moderate-scale data sets.

Model	Credit					Energy		
	Ave. loss		Ave. acc.		Average	Ave. RMSE		Average
	Train	Test	Train	Test	time (s)	Train	Test	time (s)
DNN	0.3866	0.4585	0.8365	0.8095	159.02	0.4747	0.4953	85.63
LASSO	0.4301	0.4354	0.8213	0.8187	48051.09	0.5001	0.5075	28114.84
HT-LASSO	0.4307	0.4357	0.8212	0.8192	18367.16	0.5016	0.5095	73115.23
IMP	0.4238	0.4358	0.8230	0.8171	6436.45	0.4591	0.4864	4162.50
RigL	0.4264	0.4347	0.8224	0.8172	39069.41	0.4586	0.4852	31633.43

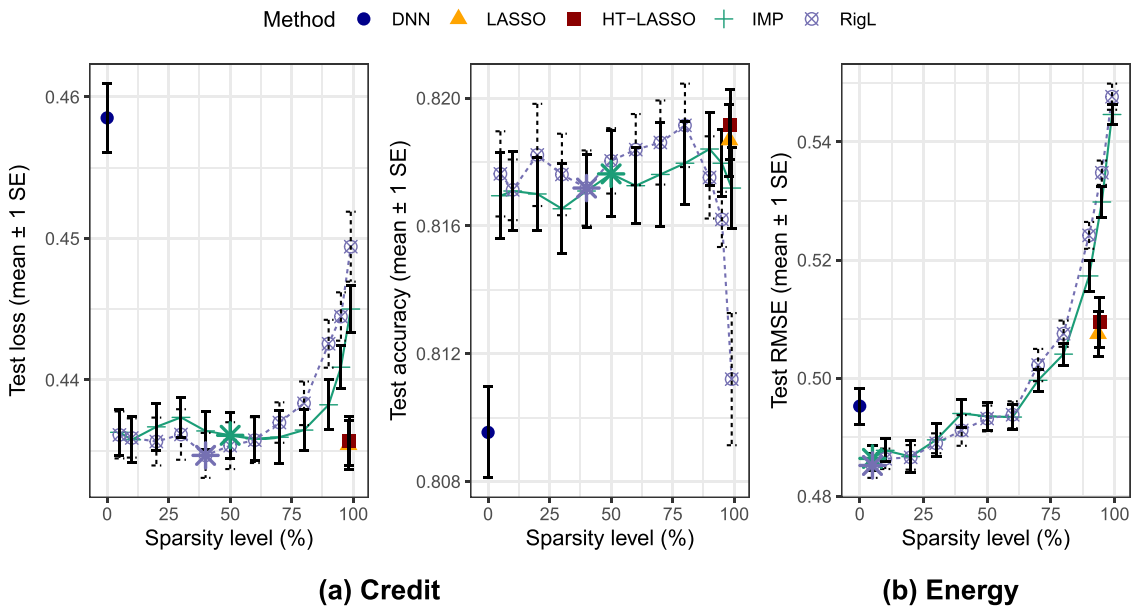


Fig. 9. Model performance plot for both moderate-scale data. The larger symbols for IMP and RigL show the sparsity level that achieves lowest MSE on validation data. Error bars show ±1 standard error.

Table 10
Summary of selected structures and cross-layer connections on two moderate-scale data.

		Average selected structures				Cross layer connections			
Data	Model	input	node	layer	link	node–node	node–output	input–node	input–output
Credit	LASSO	24	5.33	2.78	57.8	0.889	3.78	18.8	17.6
	HT-LASSO	20.4	5.33	2.78	49	0.778	3.67	14.7	13.9
Energy	LASSO	26.5	10.6	3.5	175	5.8	8.6	72.2	20.6
	HT-LASSO	26	10.6	3.4	158	4.3	9.0	60.4	18.0

accuracy or overall structural complexity. We also observe that the hard-thresholding step primarily removes additional input features while preserving a similar number of hidden nodes and layers. The right side of the table shows that both datasets retain a substantial number of direct input-to-output links. Compared with the LASSO-only results, the hard-thresholding step further reduces these cross-layer connections, resulting in a more hierarchical network.

6. Conclusions

We have proposed a general backward, node-wise pruning procedure for arbitrary feedforward neural networks and demonstrated that this pruning strategy can be effectively approximated by solving a standard GLM fitting problem. This connection provides additional transparency for pruning neural networks through shrinkage-based methods and enables fast computation by leveraging existing software for GLM optimization. Many existing studies have introduced pruning methods based on penalizing all weights in a fixed neural network architecture, such as feedforward or convolutional networks. However, because such penalized objective functions often exhibit many local optima, the resulting pruned models are typically unstable. In contrast, we find that backward node-wise pruning yields more robust and reliable solutions. In this work, we focused on the LASSO penalty due to its computational simplicity and strong ability to induce sparsity. Nonetheless, the GLM-based formulation readily extends to other shrinkage penalties, including group LASSO and related variants, thereby enabling efficient computation for a broad family of penalty functions.

Our two-step procedure simultaneously selects both the features and the structural components of a neural network, producing a stable and sparse architecture whose graphical representation reveals the functional patterns being approximated, thereby enhancing interpretability. The method begins with a dense network that includes all possible structural connections and proceeds by pruning links in a backward fashion to automatically determine the network architecture, including the relevant input features, the number of hidden layers and nodes, and the connections among them. Our simulation studies show that the soft-thresholding step based on LASSO alone cannot remove all noise variables or redundant connections, but it effectively shrinks the model space and provides a strong, sparse initialization for the subsequent hard-thresholding step. This hard-thresholding stage is essential for refining the network structure, improving stability across replicates, and achieving better predictive performance. Taken together, both steps play indispensable and complementary roles in successful model selection.

Our simulation studies in regression, binary classification, and multi-class classification settings demonstrate both the effectiveness and stability of the proposed method. The procedure not only identifies the correct features but also mitigates the overfitting commonly observed in neural network models. The resulting sparse architectures substantially improve generalizability, yielding markedly better test performance than networks trained without structural selection. In addition, the learned structures provide valuable interpretability: the selected connectivity patterns reflect key aspects of the underlying functional relationships, including the identification of important predictor interactions.

We further evaluated the method on eight small real datasets and two moderate-scale datasets, again observing strong performance in both regression and classification tasks. Across these experiments, our approach

consistently outperformed widely used unstructured pruning baselines, including Iterative Magnitude Pruning (IMP) (Frankle & Carbin, 2018) and RigL (Evci et al., 2020), achieving superior predictive accuracy at higher sparsity levels. Notably, the resulting architectures often retain direct input-to-output connections, suggesting the presence of linear components in the fitted functions that would otherwise be approximated through compositions of nonlinear transformations in standard feedforward networks.

While our proposed method is highly effective for the problems studied, extending it to very large datasets and networks presents computational challenges. Although the LASSO stage rapidly removes many connections before hard thresholding, networks with hundreds of nodes can still be computationally demanding. The LASSO step is efficient because it is applied column-wise to the weight matrix; adding a single hidden node introduces only one additional LASSO fit with $p + H$ predictors. In contrast, the hard-thresholding step must evaluate the loss after removing each remaining link in the LASSO-shrunken model. Since each additional layer (or node) may introduce up to $p + H$ new links, the procedure may require approximately $O(H^2)$ evaluations. Moreover, because the algorithm iterates until convergence, the true complexity may exceed this rate, and the computation time for the hard- evaluating the network with individual weights set to zero, and substantial speedups could be achieved by reusing intermediate computations across these evaluations.

Another direction for future research is adapting the method to other network types such as recurrent or convolutional neural networks, where the same parameter, such as a kernel or a recurrent connection, is used multiple times. A key idea in our method is to apply LASSO to blocks of non-interacting weights, to improve stability. For a convolutional network, we could apply LASSO to all parameters of all kernels at a given level. For other networks, such as those with recurrent connections, the weights do not divide so easily into sequential blocks, presenting substantially more challenges. Furthermore, while the DNN acts as a universal network without feedback connections, training a similar universal network with feedback connections would be extremely unstable, likely leading to unreliable results.

CRediT authorship contribution statement

Xinyue Zhang: Methodology, Writing – original draft; **Hong Gu:** Methodology, Writing – review & editing; **Toby Kenney:** Methodology, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary appendices

Table A.11
Regression simulation 1: Average MSEs and running time over 100 replicates for different LASSO penalty parameters and cut-off values. The top part are for the pruned models with the best λ and θ selected by the validation MSEs, together with comparisons to the DNN and the pruning baselines IMP and RigL. The bottom part reports performance immediately after the structure selection stage for different θ. Standard errors of test MSEs are reported in parentheses.

Model		Average MSE			Average time (s)
		Train	Validation	Test (SE)	
	DNN	0.9434	1.2318	1.2920 (0.0070)	44.37
	LASSO	1.0112	1.2069	1.2424 (0.0072)	364.04
	HT-LASSO	1.0397	1.1669	1.1990 (0.0066)	12103.74
	IMP	1.055	1.251	1.2579 (0.014)	866.21
	RigL	1.0252	1.2908	1.3381 (0.0113)	6090.69
$\lambda_{\min}$	LASSO model	0.9696	1.2560	1.2931 (0.0092)	159.50
	$\theta = 0$	0.9664	1.2533	1.2922 (0.0090)	6549.35
	$\theta = 0.0001$	0.9685	1.2534	1.2922 (0.0090)	2040.18
	$\theta = 0.0005$	0.9727	1.2496	1.2892 (0.0090)	2042.98
	$\theta = 0.001$	0.9768	1.2485	1.2865 (0.0090)	2025.22
	$\theta = 0.005$	1.0165	1.2027	1.2382 (0.0088)	1884.92
	$\theta = 0.01$	1.0365	1.1876	1.2106 (0.0089)	1668.65
	$\theta = 0.05$	1.0928	1.2296	1.2418 (0.0103)	1264.96
	$\theta = 0.1$	1.1424	1.2797	1.2983 (0.0125)	1120.33
$\lambda_{1\text{se}}$	LASSO model	1.0770	1.2436	1.2667 (0.0122)	171.68
	$\theta = 0$	1.0695	1.2376	1.2606 (0.0116)	1662.61
	$\theta = 0.0001$	1.0700	1.2370	1.2597 (0.0115)	711.18
	$\theta = 0.0005$	1.0713	1.2351	1.2581 (0.0116)	720.50
	$\theta = 0.001$	1.0729	1.2333	1.2570 (0.0116)	721.37
	$\theta = 0.005$	1.0834	1.2220	1.2440 (0.0118)	721.74
	$\theta = 0.01$	1.0886	1.2182	1.2357 (0.0121)	737.18
	$\theta = 0.05$	1.1063	1.2238	1.2416 (0.0123)	699.33
	$\theta = 0.1$	1.1334	1.2534	1.2695 (0.0125)	619.21

Table A.12
Regression simulation 1: Structure comparisons for different LASSO penalty and cut-off values, with standard errors (SE) reflecting variability over replicates.

Model		Average selected structures (SE)				
		input	noise	node	layer	link
	LASSO	5 (0)	2.83 (0.0451)	6.86 (0.1450)	3.41 (0.1859)	54.71 (2.8902)
	HT-LASSO	5 (0)	0.79 (0.1085)	6.53 (0.1243)	2.16 (0.1070)	28.45 (1.2968)
$\lambda_{\min}$	LASSO model	5 (0)	3 (0)	8.78 (0.1211)	6.46 (0.1466)	104.14 (1.7135)
	$\theta = 0$	5 (0)	3 (0)	8.78 (0.1211)	6.29 (0.1438)	101.64 (1.6946)
	$\theta = 0.0001$	5 (0)	2.99 (0.01)	8.77 (0.1230)	5.29 (0.1423)	85.84 (1.6777)
	$\theta = 0.0005$	5 (0)	2.99 (0.01)	8.78 (0.1211)	4.9 (0.1283)	78.74 (1.6335)
	$\theta = 0.001$	5 (0)	2.97 (0.0223)	8.77 (0.1238)	4.67 (0.1207)	73.8 (1.5536)
	$\theta = 0.005$	5 (0)	1.99 (0.0823)	8.37 (0.1195)	3.12 (0.1037)	44.89 (1.0052)
	$\theta = 0.01$	5 (0)	0.85 (0.0903)	7.55 (0.1019)	2.39 (0.0709)	31.74 (0.5745)
	$\theta = 0.05$	5 (0)	0.09 (0.0288)	5.49 (0.0927)	1.58 (0.0554)	19.28 (0.3464)
	$\theta = 0.1$	5 (0)	0.04 (0.0197)	4.52 (0.0759)	1.27 (0.0509)	15.76 (0.2807)
λ_{1se}	LASSO model	5 (0)	2.71 (0.0574)	5.92 (0.1390)	2.46 (0.0947)	37.07 (1.0798)
	$\theta = 0$	5 (0)	2.7 (0.0577)	6.92 (0.1390)	2.37 (0.0950)	36.34 (1.0567)
	$\theta = 0.0001$	5 (0)	2.51 (0.0674)	5.92 (0.1390)	2.22 (0.0871)	34.24 (0.9698)
	$\theta = 0.0005$	5 (0)	2.23 (0.0863)	5.92 (0.1390)	2.16 (0.0849)	32.21 (0.9058)
	$\theta = 0.001$	5 (0)	1.98 (0.0974)	5.92 (0.1390)	2.17 (0.0865)	30.78 (0.8622)
	$\theta = 0.005$	5 (0)	0.8 (0.0853)	5.92 (0.1390)	1.96 (0.0803)	25.28 (0.6054)
	$\theta = 0.01$	5 (0)	0.43 (0.0671)	5.91 (0.1379)	1.83 (0.0682)	23.33 (0.5117)
	$\theta = 0.05$	5 (0)	0.08 (0.0339)	5.48 (0.1185)	1.56 (0.0592)	19.51 (0.3836)
	$\theta = 0.1$	5 (0)	0.04 (0.0243)	4.83 (0.0933)	1.38 (0.0528)	16.98 (0.2923)

Table A.13
Regression simulation 2: Average MSEs and running time over 100 replicates for different methods with different values for H.

H	Model	Average MSE			Average time (s)
		Train	Validation	Test (SE)	
H = 10	DNN	4.0098	4.3059	4.2857 (0.0093)	90.03
	LASSO	4.0262	4.2903	4.2642 (0.0077)	1045.42
	HT-LASSO	4.0718	4.2431	4.2264 (0.0071)	4758.83
	IMP	4.1312	4.4386	4.381 (0.014)	2065.57
	RigL	4.1161	4.4121	4.3781 (0.0192)	21689.43
H = 20	DNN	3.7878	4.3966	4.3645 (0.0088)	120.69
	LASSO	4.0207	4.3122	4.2780 (0.0136)	1480.99
	HT-LASSO	4.0948	4.2623	4.2234 (0.0059)	4750.73
	IMP	3.9977	4.3362	4.2753 (0.0142)	3061.69
	RigL	3.9142	4.3562	4.314 (0.0063)	38832.77
H = 30	DNN	3.5451	4.5309	4.5482 (0.0164)	154.16
	LASSO	3.9926	4.3490	4.3221 (0.0207)	2228.10
	HT-LASSO	4.1001	4.2630	4.2493 (0.0151)	5793.66
	IMP	3.986	4.2787	4.2535 (0.0118)	4264.84
	RigL	3.8737	4.3705	4.3519 (0.0109)	60108.795

Table A.14
Simulation on binary classification: Average entropy loss , accuracy, and running time over 100 replicates for different LASSO penalty parameters and cut-off values.

Model		Average loss			Average accuracy			Average time (s)
		Train	Validation	Test (SE)	Train	Validation	Test (SE)	
	DNN	0.4920	0.5450	0.5481 (0.0006)	0.7532	0.7205	0.7189 (0.0003)	29.51
	LASSO	0.5169	0.5365	0.5361 (0.0005)	0.7372	0.7240	0.7258 (0.0004)	710.34
	HT-LASSO	0.5217	0.5325	0.5321 (0.0004)	0.7345	0.7270	0.7282 (0.0002)	2223.34
	IMP	0.5211	0.5423	0.5414 (0.0003)	0.7363	0.7197	0.7213 (0.0003)	902.84
	RigL	0.5220	0.5405	0.5400 (0.0004)	0.7360	0.7219	0.7225 (0.0003)	3022.62
$\lambda_{\min}$	LASSO model	0.4929	0.5484	0.5494 (0.0010)	0.7522	0.7173	0.7179 (0.0006)	458.59
	$\theta = 0$	0.4911	0.5488	0.5500 (0.0009)	0.7534	0.7174	0.7179 (0.0004)	2092.93
	$\theta = 0.0001$	0.4916	0.5485	0.5497 (0.0009)	0.7530	0.7182	0.7180 (0.0004)	654.33
	$\theta = 0.0005$	0.4926	0.5479	0.5492 (0.0009)	0.7524	0.7183	0.7182 (0.0004)	639.83
	$\theta = 0.001$	0.4955	0.5467	0.5479 (0.0009)	0.7513	0.7197	0.7187 (0.0005)	616.68
	$\theta = 0.005$	0.5252	0.5379	0.5375 (0.0009)	0.7319	0.7240	0.7244 (0.0006)	335.75
	$\theta = 0.01$	0.5433	0.5521	0.5517 (0.0019)	0.7193	0.7138	0.7145 (0.0016)	254.42
	$\theta = 0.05$	0.6556	0.6565	0.6561 (0.0012)	0.6327	0.6314	0.6321 (0.0013)	81.11
	$\theta = 0.1$	0.6598	0.6598	0.6598 (0.0001)	0.6283	0.6284	0.6283 (0.0002)	73.14
λ_{1se}	LASSO model	0.5234	0.5376	0.5359 (0.0005)	0.7342	0.7245	0.7266 (0.0003)	227.49
	$\theta = 0$	0.5226	0.5371	0.5355 (0.0004)	0.7348	0.7250	0.7269 (0.0003)	66.75
	$\theta = 0.0001$	0.5227	0.5370	0.5354 (0.0004)	0.7348	0.7252	0.7269 (0.0003)	54.46
	$\theta = 0.0005$	0.5229	0.5371	0.5354 (0.0004)	0.7346	0.7248	0.7270 (0.0003)	53.94
	$\theta = 0.001$	0.5231	0.5368	0.5353 (0.0004)	0.7346	0.7247	0.7271 (0.0003)	54.17
	$\theta = 0.005$	0.5252	0.5355	0.5343 (0.0005)	0.7334	0.7257	0.7277 (0.0003)	51.80
	$\theta = 0.01$	0.5297	0.5394	0.5381 (0.0009)	0.7308	0.7229	0.7253 (0.0005)	53.18
	$\theta = 0.05$	0.6421	0.6423	0.6432 (0.0029)	0.6460	0.6462	0.6459 (0.0030)	30.25
	$\theta = 0.1$	0.6566	0.6570	0.6572 (0.0012)	0.6318	0.6314	0.6313 (0.0013)	24.53

Table A.15
Simulation on binary classification: Structure comparisons for different LASSO penalty and cut-off values, with standard errors (SE) reflecting variability over replicates.

Model		Average selected structures (SE)				
		input	noise	node	layer	link
	LASSO	5 (0)	1.94 (0.1090)	3.57 (0.1565)	1.94 (0.1332)	26.06 (2.0913)
	HT-LASSO	5 (0)	0.28 (0.0668)	3.28 (0.1147)	1.41 (0.0637)	13.56 (0.6112)
$\lambda_{\min}$	LASSO model	5 (0)	3 (0)	7.86 (0.1429)	5.72 (0.1564)	88.04 (2.0136)
	$\theta = 0$	5 (0)	3 (0)	7.86 (0.1429)	5.63 (0.1600)	86.73 (1.9907)
	$\theta = 0.0001$	5 (0)	3 (0)	7.86 (0.1429)	5.21 (0.1533)	78.43 (1.8278)
	$\theta = 0.0005$	5 (0)	2.98 (0.0141)	7.86 (0.1429)	4.87 (0.1502)	71.04 (1.8247)
	$\theta = 0.001$	5 (0)	2.9 (0.0389)	7.86 (0.1429)	4.61 (0.1497)	62.89 (1.6856)
	$\theta = 0.005$	4.9 (0.0362)	0.6 (0.0752)	5.32 (0.1262)	2.05 (0.0783)	18.09 (0.5257)
	$\theta = 0.01$	3.93 (0.0924)	0.13 (0.0338)	3.87 (0.1143)	1.57 (0.0640)	11.26 (0.3145)
	$\theta = 0.05$	0.2 (0.0492)	0 (0)	0.33 (0.0779)	0.15 (0.0359)	0.56 (0.1373)
	$\theta = 0.1$	0.08 (0.0298)	0 (0)	0.09 (0.0172)	0.02 (0.0181)	0.18 (0.0912)
λ_{1se}	LASSO model	5 (0)	1.65 (0.1038)	3.72 (0.0911)	1.34 (0.0639)	15.74 (0.5544)
	$\theta = 0$	5 (0)	1.6 (0.1064)	2.72 (0.0911)	1.34 (0.0639)	15.47 (0.5400)
	$\theta = 0.0001$	5 (0)	1.45 (0.1009)	2.72 (0.0911)	1.33 (0.0620)	15.07 (0.5207)
	$\theta = 0.0005$	5 (0)	1.17 (0.0985)	2.72 (0.0911)	1.31 (0.0615)	14.38 (0.4976)
	$\theta = 0.001$	5 (0)	0.87 (0.0917)	2.72 (0.0911)	1.28 (0.0533)	13.8 (0.4651)
	$\theta = 0.005$	4.95 (0.0261)	0.14 (0.0377)	2.72 (0.0911)	1.21 (0.0433)	11.56 (0.2783)
	$\theta = 0.01$	4.55 (0.0702)	0.03 (0.0171)	2.71 (0.0913)	1.2 (0.0426)	10.46 (0.2572)
	$\theta = 0.05$	0.6 (0.0985)	0 (0)	0.73 (0.1072)	0.32 (0.0490)	1.59 (0.2523)
	$\theta = 0.1$	0.1 (0.0414)	0 (0)	0.13 (0.0525)	0.06 (0.0239)	0.27 (0.1109)

Table A.16
Simulation on three-label classification: Average cross-entropy loss , accuracy, and running time over 100 replicates for different LASSO penalty parameters and cut-off values.

Model		Average loss			Average accuracy			Average time (s)
		Train	Validation	Test (SE)	Train	Validation	Test (SE)	
	DNN	0.6065	0.6816	0.6907 (0.0011)	0.7144	0.6739	0.6728 (0.0005)	30.66
	LASSO	0.6119	0.6782	0.6825 (0.0013)	0.7118	0.6754	0.6755 (0.0006)	923.30
	HT-LASSO	0.6429	0.6607	0.6639 (0.0006)	0.6924	0.6831	0.6817 (0.0004)	8533.56
	IMP	0.6557	0.6922	0.6916 (0.0008)	0.6954	0.6726	0.6742 (0.0006)	918.37
	RigL	0.6512	0.6898	0.6881 (0.0007)	0.6963	0.6723	0.6755 (0.0005)	3608.46
$\lambda_{\min}$	LASSO model	0.6161	0.6782	0.6825 (0.0013)	0.7092	0.6755	0.6755 (0.0006)	636.52
	$\theta = 0$	0.6136	0.6785	0.6835 (0.0014)	0.7103	0.6760	0.6751 (0.0006)	3602.25
	$\theta = 0.0001$	0.6145	0.6785	0.6832 (0.0014)	0.7102	0.6755	0.6752 (0.0006)	881.44
	$\theta = 0.0005$	0.6159	0.6780	0.6826 (0.0014)	0.7094	0.6751	0.6754 (0.0006)	844.32
	$\theta = 0.001$	0.6179	0.6772	0.6820 (0.0014)	0.7084	0.6756	0.6756 (0.0006)	827.87
	$\theta = 0.005$	0.6487	0.6675	0.6696 (0.0008)	0.6890	0.6793	0.6791 (0.0005)	639.48
	$\theta = 0.01$	0.6646	0.6764	0.6779 (0.0015)	0.6818	0.6762	0.6750 (0.0008)	546.96
	$\theta = 0.05$	0.8752	0.8827	0.8825 (0.0143)	0.5412	0.5369	0.5380 (0.0082)	324.99
	$\theta = 0.1$	1.0566	1.0562	1.0576 (0.0072)	0.4226	0.4237	0.4230 (0.0057)	138.85
λ_{1se}	LASSO model	0.7127	0.7188	0.7205 (0.0023)	0.6681	0.6642	0.6639 (0.0015)	251.25
	$\theta = 0$	0.7094	0.7150	0.7169 (0.0008)	0.6706	0.6675	0.6664 (0.0004)	64.68
	$\theta = 0.0001$	0.7094	0.7150	0.7169 (0.0008)	0.6706	0.6670	0.6663 (0.0004)	63.37
	$\theta = 0.0005$	0.7096	0.7149	0.7169 (0.0008)	0.6704	0.6673	0.6664 (0.0004)	62.13
	$\theta = 0.001$	0.7098	0.7146	0.7167 (0.0008)	0.6703	0.6668	0.6664 (0.0004)	62.84
	$\theta = 0.005$	0.7111	0.7149	0.7171 (0.0008)	0.6699	0.6678	0.6669 (0.0004)	65.08
	$\theta = 0.01$	0.7146	0.7188	0.7211 (0.0013)	0.6678	0.6644	0.6650 (0.0009)	65.47
	$\theta = 0.05$	0.8343	0.8359	0.8387 (0.0039)	0.5671	0.5669	0.5670 (0.0032)	63.49
	$\theta = 0.1$	0.8557	0.8573	0.8593 (0.0060)	0.5526	0.5535	0.5533 (0.0032)	57.16

Table A.17
Simulation on three-label classification: Structure comparisons for different LASSO penalty and cut-off values, with standard errors (SE) reflecting variability over replicates.

Model		Average selected structures (SE)				
		input	noise	node	layer	link
	LASSO	5 (0)	3 (0)	7.9 (0.2025)	5.54 (0.1963)	99.76 (2.7335)
	HT-LASSO	5 (0)	0.34 (0.1129)	6.72 (0.1917)	2.8 (0.1178)	30.7 (1.1372)
$\lambda_{\min}$	LASSO model	5 (0)	3 (0)	10.9 (0.2025)	5.54 (0.1963)	99.76 (2.7335)
	$\theta = 0$	5 (0)	3 (0)	7.9 (0.2025)	5.42 (0.1981)	97.82 (2.6614)
	$\theta = 0.0001$	5 (0)	3 (0)	7.9 (0.2025)	4.62 (0.1805)	85.84 (2.3864)
	$\theta = 0.0005$	5 (0)	2.98 (0)	7.9 (0.2025)	4.46 (0.1789)	77.7 (2.3902)
	$\theta = 0.001$	5 (0)	2.9 (0.0429)	7.88 (0.2033)	4.32 (0.1861)	71.52 (2.3516)
	$\theta = 0.005$	5 (0)	0.26 (0.0689)	6.86 (0.2041)	2.98 (0.1471)	31.2 (0.9773)
	$\theta = 0.01$	5 (0)	0.08 (0.0481)	5.58 (0.1616)	2.38 (0.1026)	21.64 (0.6495)
	$\theta = 0.05$	2.82 (0.2073)	0.02 (0.02)	2.54 (0.2232)	1.22 (0.1042)	7.08 (0.5641)
	$\theta = 0.1$	0.32 (0.0725)	0 (0)	0.1 (0.0515)	0.06 (0.0444)	0.5 (0.1571)
λ_{1se}	LASSO model	5 (0)	1.38 (0.1242)	1.3 (0.0870)	1.28 (0.0810)	15.96 (0.2303)
	$\theta = 0$	5 (0)	1.3 (0.1317)	1.3 (0.0870)	1.28 (0.0810)	15.88 (0.2350)
	$\theta = 0.0001$	5 (0)	1.2 (0.1212)	1.3 (0.0870)	1.28 (0.0810)	15.74 (0.2282)
	$\theta = 0.0005$	5 (0)	0.92 (0.1137)	1.3 (0.0870)	1.28 (0.0810)	15.28 (0.2214)
	$\theta = 0.001$	5 (0)	0.56 (0.0865)	1.3 (0.0870)	1.28 (0.0810)	14.78 (0.2006)
	$\theta = 0.005$	5 (0)	0.04 (0.0280)	1.14 (0.0858)	1.28 (0.0810)	13.7 (0.1943)
	$\theta = 0.01$	5 (0)	0 (0)	1.3 (0.1079)	1.28 (0.0810)	12.76 (0.24)
	$\theta = 0.05$	3.1 (0.0655)	0.02 (0.02)	1.3 (0.1143)	1.28 (0.0810)	6.56 (0.2141)
	$\theta = 0.1$	2.82 (0.0739)	0 (0)	1.18 (0.0899)	1.18 (0.0889)	5.84 (0.1945)

Table A.18
Effect of random seed on pruning.

Average MSE			
Model	Train	Validation	Test
DNN	0.9475 (0.0042)	1.3325 (0.0855)	1.3197 (0.0105)
LASSO	1.0147 (0.0092)	1.2895 (0.0775)	1.2705 (0.0094)
HT-LASSO	1.0461 (0.0098)	1.2442 (0.0715)	1.2325 (0.0080)

Average selected structures					
Model	input	noise	node	layer	links
LASSO	5 (0.00)	2.71 (0.08)	6.62 (0.18)	3.28 (0.20)	51.56 (3.15)
HT-LASSO	5 (0.00)	0.67 (0.12)	6.19 (0.17)	1.95 (0.10)	26.75 (1.32)

References

Alemu, H. Z., Zhao, J., Li, F., & Wu, W. (2019). Group l1/2 regularization for pruning hidden layer nodes of feedforward neural networks. *IEEE Access*, *7*, 9540–9557. https://ieeexplore.ieee.org/document/8600728.

Asuncion, A., & Newman, D. (2007). Uci Machine Learning Repository. University of California, Irvine. Available at: https://archive.ics.uci.edu.

Augasta, M., & Kathirvalavakumar, T. (2013). Pruning algorithms of neural networks—a comparative study. *Open Computer Science*, *3*(3), 105–115.

Bao, Y., Liu, Z., Luo, Z., & Yang, S. (2022). Smooth group l1/2 regularization for pruning convolutional neural networks. *Symmetry*, *14*(1), 154. https://doi.org/10.3390/sym14010154

Bebis, G., & Georgiopoulos, M. (1994). Feed-forward neural networks. *IEEE Potentials*, *13*(4), 27–31.

Evci, U., Gale, T., Menick, J., Castro, P. S., & Elsen, E. (2020). Rigging the lottery: Making all tickets winners. In *International conference on machine learning* (pp. 2943–2952). PMLR.

Fan, J., & Li, R. (2001). Variable selection via nonconcave penalized likelihood and its oracle properties. *Journal of the American Statistical Association*, *96*(456), 1348–1360.

Fan, Y., Tao, B., Zheng, Y., & Jang, S.-S. (2019). A data-driven soft sensor based on multilayer perceptron neural network with a double lasso approach. *IEEE Transactions on Instrumentation and Measurement*, *69*(7), 3972–3979.

Feng, J., & Simon, N. (2017). Sparse-input neural networks for high-dimensional nonparametric regression and classification. arXiv:1711.07592. Available at: https://arxiv.org/abs/1711.07592.

Frankle, J., & Carbin, M. (2018). The lottery ticket hypothesis: Finding sparse, trainable neural networks. arXiv:1803.03635 Available at: https://arxiv.org/abs/1803.03635.

Giles, C. L., & Maxwell, T. (1987). Learning, invariance, and generalization in high-order neural networks. *Applied Optics*, *26*(23), 4972–4978.

Gui, J., Sun, Z., Ji, S., Tao, D., & Tan, T. (2016). Feature selection based on structured sparsity: A comprehensive study. *IEEE Transactions on Neural Networks and Learning Systems*, *28*(7), 1490–1507.

Haykin, S. (1999). Neural networks: A comprehensive foundation 2nd ed.. Prentice-Hall Of India Pvt. Limited. https://books.google.ca/books?id=5YO3jwEACAAJ.

Huang, G., Liu, Z., Maaten, L. V. D., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the ieee conference on computer vision and pattern recognition* (pp. 4700–4708).

Karnin, E. D. (1990). A simple procedure for pruning back-propagation trained neural networks. *IEEE Transactions on Neural Networks*, *1*(2), 239–242.

Lemhadri, I., Ruan, F., & Tibshirani, R. (2021). Lassonet: Neural networks with feature sparsity. In *International conference on artificial intelligence and statistics* (pp. 10–18). PMLR.

Liu, B., Zhang, Z., He, P., Wang, Z., Xiao, Y., Ye, R., Zhou, Y., Ku, W.-S., & Hui, B. (2024). A survey of lottery ticket hypothesis. arXiv:2403.04861. Available at: https://arxiv.org/abs/2403.04861.

Liu, X., Li, M., Li, X., Qu, L., Peng, Z., Song, Y., Liu, Z., Jiang, L., & Li, J. (2025). Automatic pruning via structured lasso with class-wise information. arXiv:2502.09125. Available at: https://arxiv.org/abs/2502.09125v1.

Medsker, L. R., & Jain, L. (2001). Recurrent neural networks. *Design and Applications*, *5*, 64–67.

Mocanu, D. C., Mocanu, E., Stone, P., Nguyen, P. H., Gibescu, M., & Liotta, A. (2018). Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature Communications*, *9*(1), 2383.

Molchanov, P., Mallya, A., Tyree, S., Frosio, I., & Kautz, J. (2019). Importance estimation for neural network pruning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 11264–11272).

O'Shea, K., & Nash, R. 2015. An introduction to convolutional neural networks. arXiv:1511.08458. Available at: https://arxiv.org/abs/1511.08458.

Oyedotun, O., Aouada, D., & Ottersten, B. (2020). Structured compression of deep neural networks with debiased elastic group lasso. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision* (pp. 2277–2286).

Pham, H., Liu, S., Xiang, L., Le, D., Wen, H., Tran-Thanh, L. et al. (2023). Towards data-agnostic pruning at initialization: What makes a good sparse mask? *Advances in Neural Information Processing Systems*, *36*, 80044–80065.

Rajaraman, N., Devvrit, F., Mokhtari, A., & Ramchandran, K. (2023). Greedy pruning with group lasso provably generalizes for matrix sensing. *Advances in Neural Information Processing Systems*, *36*, 60117–60129.

Sabo, D., & Yu, X.-H. (2008). A new pruning algorithm for neural network dimension analysis. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)* (pp. 3313–3318). IEEE.

Sanh, V., Wolf, T., & Rush, A. (2020). Movement pruning: Adaptive sparsity by fine-tuning. *Advances in Neural Information Processing Systems*, *33*, 20378–20389.

Scardapane, S., Comminiello, D., Hussain, A., & Uncini, A. (2016). Group sparse regularization for deep neural networks. arXiv:1607.00485. Available at: https://arxiv.org/abs/1607.00485.

Shao, Y., Zhao, K., Cao, Z., Peng, Z., Peng, X., Li, P., Wang, Y., & Ma, J. (2022). Mobileprune: Neural network compression via l0 sparse group lasso on the mobile system. *Sensors*, *22*(11), 4081.

Shi, Y., Tang, A., Niu, L., & Zhou, R. (2024). Sparse optimization guided pruning for neural networks. *Neurocomputing*, *574*, 127280.

Sun, K., Huang, S.-H., Wong, D. S.-H., & Jang, S.-S. (2016). Design and application of a variable selection method for multilayer perceptron neural network with lasso. *IEEE Transactions on Neural Networks and Learning Systems*, *28*(6), 1386–1396.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, *58*(1), 267–288.

Wang, H., Qin, C., Bai, Y., Zhang, Y., & Fu, Y. (2021). Recent advances on neural network pruning at initialization. arXiv:2103.06460. Available at: https://arxiv.org/abs/2103.06460.

Wang, H., Sui, L., Zhang, M., Zhang, F., Ma, F., & Sun, K. (2021b). A novel input variable selection and structure optimization algorithm for multilayer perceptron-based soft sensors. *Mathematical Problems in Engineering*, *2021*, 1–10.

Wang, J., Chang, Q., Liu, Y., & Pal, N. R. (2019). Weight noise injection-based mlps with group lasso penalty: Asymptotic convergence and application to node pruning. *IEEE Transactions on Cybernetics*, *49*(12), 4346–4364. https://doi.org/10.1109/TCYB.2018.2864142

Wang, J., Xu, C., Yang, X., & Zurada, J. M. (2017). A novel pruning algorithm for smoothing feedforward neural networks based on group lasso method. *IEEE Transactions on Neural Networks and Learning Systems*, *29*(5), 2012–2024.

Wang, J., Xu, C., Yang, X., & Zurada, J. M. (2018). A novel pruning algorithm for smoothing feedforward neural networks based on group lasso method. *IEEE Transactions on Neural Networks and Learning Systems*, *29*(5), 2012–2024. https://doi.org/10.1109/TNNLS.2017.2748585

Wen, C., Wang, X., & Wang, S. (2015). Laplace error penalty-based variable selection in high dimension. *Scandinavian Journal of Statistics*, *42*(3), 685–700.

Zhang, C.-H. (2010). Nearly unbiased variable selection under minimax concave penalty. *The Annals of Statistics*, *38*(2), 894–942.

Zhang, H., Wang, J., Sun, Z., Zurada, J. M., & Pal, N. R. (2019). Feature selection for neural networks using group lasso regularization. *IEEE Transactions on Knowledge and Data Engineering*, *32*(4), 659–673.

Zhao, H., Guan, R., Man, K. L., Yu, L., & Yue, Y. (2025). Repair: Repaired pruning at initialization resilience. *Neural Networks*, *184*, 107086.